

Тема лекции

Управление ФС, памятью, реестром и процессами.

План лекции.

- Стандартные элементы управления.
- Управление ФС при помощи функций Windows.

Цели лекции.

Ознакомится с методами и способами управления ФС, памятью, реестром и процессами посредством WinAPI.

Теоретическая часть лекции.

Стандартные элементы управления.

Окна и элементы управления

Кроме главного окна, приложение может создавать и другие окна: перекрываемые WS_OVERLAPPED, всплывающие WS_POPUP, дочерние WS_CHILD. *Каждое окно должно иметь свою оконную функцию, куда передаются посылаемые операционной системой сообщения.*

Начнем рассмотрение с создания дочерних окон, особенностью которых является то, что они располагаются в клиентской области родительского окна и автоматически уничтожаются при его закрытии.

Для создания окна используется функция CreateWindow() или ее расширение CreateWindowEx():

`HWND WINAPI CreateWindowExW(DWORD dwExStyle, LPCWSTR lpClassName, LPCWSTR lpWindowName, DWORD dwStyle, int X, int Y, int nWidth, int nHeight, HWND hWndParent, HMENU hMenu, HINSTANCE hInstance, LPVOID lpParam);`

Функции отличаются лишь первым параметром расширенных стилей dwExStyle. Если посмотреть файл включений winuser.h, то можно увидеть, что все вызовы функции CreateWindow() заменяются ее расширенной версией, где первый параметр имеет нулевое значение.

ПРИМЕЧАНИЕ

Параметр hMenu используется для задания уникального идентификатора дочернего окна child_ID. В этом случае приходится задавать явное преобразование типа (HMENU)child_ID.

В главе 1 мы уже рассматривали значение аргументов этой функции при анализе "скелета" Windows-приложения. Вспомним последовательность шагов при создании окна:

1. Создается переменная типа "Класс окна" WNDCLASSEX.
2. Регистрируется класс окна функцией RegisterClassEx().
3. Вызывается функция CreateWindowEx() для создания окна.
4. При помощи функции ShowWindow() окно отображается.
5. Поскольку отображение окна происходит асинхронно, то обычно используют функцию UpdateWindow() для принудительной прорисовки окна.

ПРИМЕЧАНИЕ

Сейчас, как правило, используют расширенный класс окна WNDCLASSEX вместо WNDCLASS и, соответственно, расширенную функцию регистрации класса окна RegisterClassEx().

Рассмотрим процесс создания дочерних окон на примере реализации нескольких учебных задач.

Дочерние окна

Построим приложение, имитирующее широко известную игру "крестики-нолики", где вторым партнером будет выступать компьютер (рис. 3.1). В качестве заготовки используем стандартный Win32-проект. Добавим один пункт меню **New** для запуска новой игры. Текст оконной функции приложения приведен в листинге 3.1.

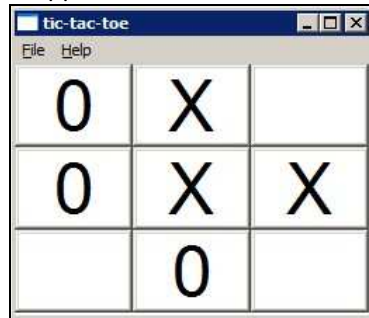


Рис. 3.1. Крестики-нолики

Основная идея, которая реализуется в этой программе, заключается в том, что вместо рисования прямоугольников и вычисления затем позиции нажатия левой кнопки мыши, мы создадим 9 дочерних окон и будем обрабатывать сообщение о щелчке мышью в соответствующем окне.

Листинг 3.1. Крестики-нолики

```
LRESULT CALLBACK ChildProc(HWND, UINT, WPARAM, LPARAM);
TCHAR ChildClassName[MAX_LOADSTRING] = _T("WinChild");
ATOM MyRegisterChildClass()
{
    WNDCLASSEX wcex = { 0 };
    wcex.cbSize = sizeof(WNDCLASSEX);
    wcex.lpfnWndProc = ChildProc;
    wcex.hInstance = hInst;
    wcex.hCursor = LoadCursor(NULL, IDC_ARROW);
    wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW+1);
    wcex.lpszClassName = ChildClassName; return
    RegisterClassEx(&wcex);
} static HFONT newFont; static HWND hChild[9];
unsigned char k[9] = { 0 }; char text[] = { ' ', 'X', 'O' };
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM
IParam) { int i; static int sx, sy; switch (message)
{
    case WM_CREATE:
        MyRegisterChildClass(); for (i = 0; i < 9; i++)
            hChild[i] = CreateWindow(ChildClassName, NULL, WS_CHILD | WS_DLGFRAME |
WS_VISIBLE, 0, 0, 0, 0, hWnd, NULL, hInst, NULL); break; case WM_SIZE:
    if (wParam == SIZE_MINIMIZED) break; //Кнопка свертывания окна sx =
    LOWORD(IParam)/3; //Ширина дочернего окна sy = HIWORD(IParam)/3; //Высота
    дочернего окна if (newFont) DeleteObject(newFont);
```

```

        newFont = CreateFont(min(sx,sy), 0, 0, 0, FW_NORMAL, 0, 0, 0,  DEFAULT_CHARSET,
OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,  DEFAULT_QUALITY, DEFAULT_PITCH |
FF_DONTCARE, _T("Arial"));  for (i = 0; i < 9; i++)
    { MoveWindow(hChild[i],(i%3)*sx, (i/3)*sy, sx, sy, TRUE);
      UpdateWindow(hChild[i]);
    }
break; case WM_COMMAND:
switch (LOWORD(wParam))
{
case ID_NEW:
    for (i = 0; i < 9; i++)
    {
        k[i] = 0;
        InvalidateRect(hChild[i], NULL, 1);
        UpdateWindow(hChild[i]);
    }
    break;
    case IDM_EXIT: DestroyWindow(hWnd); break;
default: return DefWindowProc(hWnd, message, wParam, lParam);
}
break;
case WM_DESTROY: PostQuitMessage(0); break; default: return
DefWindowProc(hWnd, message, wParam, lParam);
}
return 0;
}
////////////////////////////////////
LRESULT CALLBACK ChildProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    PAINTSTRUCT ps;
    HDC hdc; RECT rt; int i, s; char
*ch; switch (message)
    {
        case WM_LBUTTONDOWN :
            for (i = 0; hWnd != hChild[i]; i++);  if (k[i]) break; else k[i] = 1;
            InvalidateRect(hWnd, NULL, 1);  UpdateWindow(hWnd);
srand(lParam);  for(i = s = 0; i < 9; i++) if (k[i]) s++;
            if(s == 9) MessageBox(hWnd, _T("Для следующего сеанса выберите\
New"),
_T("Конец игры"), MB_OK | MB_ICONQUESTION);
            else
            {
                while(true)
                { s = rand()*9/(RAND_MAX+1);  if (k[s]) continue;
k[s] = 2;
                InvalidateRect(hChild[s], NULL, 1);  UpdateWindow(hChild[s]);
                break;
            }
            }
            break; case WM_PAINT: for (i
= 0; hWnd != hChild[i]; i++);  if(k[i])
            {

```

```

        ch = text+k[i];   hdc = BeginPaint(hWnd, &ps);
GetClientRect(hWnd, &rt);
        SelectObject(hdc, newFont);
        DrawTextA(hdc,ch,1,&rt,DT_SINGLELINE|DT_CENTER|DT_VCENTER);
        EndPaint(hWnd, &ps); } //Фоновая закраска окна   else
DefWindowProc(hWnd, message, wParam, lParam); break;
        default: return DefWindowProc(hWnd, message, wParam, lParam);
    } return 0;
}

```

При создании главного окна приложения в сообщении WM_CREATE создадим 9 дочерних окон. Для всех дочерних окон будем использовать один класс окна и, соответственно, одну оконную функцию ChildProc. В локальной функции MyRegisterChildClass() определим класс окна. Поскольку многие поля переменной нужно оставить со значением по умолчанию, обнулим всю структуру при описании и заполним лишь необходимые поля. После чего зарегистрируем класс окна с именем "WinChild".

Так как при обработке сообщения WM_CREATE главное окно приложения еще не прорисовано, дочерние окна создаем нулевого размера. Укажем стиль WS_VISIBLE для того, чтобы окно было отображаемым. Стиль рамки WS_DLGFAME запретит изменение размеров окна при буксировке его границ.

Определим на глобальном уровне массив типа HWND дескрипторов дочерних окон hChild[9], дескриптор шрифта newFont, а также статический массив k[9] типа unsigned char, где мы будем хранить признак заполнения окна:

- ☐ 0 — окно не занято;
- ☐ 1 — пользователь щелкнул левой кнопкой мыши по этому окну, 'X';
- ☐ 2 — компьютер выбрал это окно для вывода, 'O'.

Для корректировки размера дочерних окон используем обработчик сообщения WM_SIZE, поскольку это сообщение генерируется при любом изменении размера главного окна.

Если сообщение связано с кнопкой минимизации и wParam == SIZE_MINIMIZED, перерисовывать окно нет необходимости, и мы заканчиваем обработку сообщения оператором break.

Ширину и высоту главного окна получим из параметра младшего и старшего слова lParam. Размеры дочерних окон установим в 1/3 от полученного размера.

Создадим шрифт, определив его размер по размеру созданного окна. Дескриптор шрифта определен на глобальном уровне, он будет доступен и в оконной функции дочернего окна. Однако при изменении размера окна мы создаем новый шрифт, предварительно удаляя текущий, если он уже определен.

В цикле по переменной i нарисуем 9 окон функцией MoveWindow():

```

BOOL WINAPI MoveWindow(HWND hWnd, int x, int y, int nWidth, int nHeight, BOOL bRepaint);

```

Функция переопределяет позицию вывода окна (x,y) и его размеры nWidth, nHeight. Если последний параметр отличен от 0, генерируется сообщение WM_PAINT для перерисовки окна.

Координаты левого верхнего угла окна определяются формулами:

```

x = (i%3)*sx; y = (i/3)*sy;

```

Действительно, в клиентской области главного окна координату x можно вычислить как остаток от деления i на 3, умноженный на ширину дочернего окна; а координату y как целую часть от деления i на 3, умноженную на высоту дочернего окна.

Здесь, однако, необходимо использовать функцию UpdateWindow(), иначе окна не успеют перерисовываться.

Для того чтобы иметь возможность запустить новый сеанс игры, добавим в меню программы пункт **New** с идентификатором ID_NEW. В обработчике этой команды очистим массив k и перерисуем все дочерние окна.

Основная же логика задачи сосредоточена в оконной функции дочерних окон. Мы будем использовать одну функцию для всех 9 окон, а различать их будем по дескрипторам, которые сохранили в массиве `hChild` на глобальном уровне.

При обработке сообщения `WM_LBUTTONDOWN` сначала мы должны определить индекс окна, пославшего сообщение. Это можно сделать, организовав в цикле перебор дескрипторов дочерних окон и сравнив их с дескриптором окна, пославшего сообщение: `for (i = 0; hWnd != hChild[i]; i++)`;

По завершении цикла переменная `i` и будет искомым индексом.

Если значение элемента массива `k[i]` отлично от нуля, мы игнорируем эту операцию, прекращая обработку сообщения оператором `break`. Если же окно свободно, присваиваем элементу массива `k[i]` значение 1.

После этого нужно перерисовать дочернее окно последовательным вызовом функций `InvalidateRect()` и `UpdateWindow()`.

Сейчас компьютер должен выбрать окно, куда он выведет ответный "нолик". Для этого мы используем функцию генерации случайного числа. Однако функция будет генерировать одну и ту же последовательность случайных чисел, что нас вряд ли устроит. Для того чтобы внести некоторое разнообразие в выборе ответа, мы ввели функцию `srand()`, которая задает различные числовые последовательности, но их выбор опять же зависит от ее аргумента, в качестве которого мы взяли `IParam`, составленный из координат курсора мыши. Так что, щелкая мышью в разных частях окна, мы получим разные последовательности случайных чисел. Прием, конечно, примитивный, но действенный.

Однако предварительно проверим, не все ли окна уже заняты? Просто подсчитаем количество ненулевых значений массива `k` и, если это количество равно 9, выведем окно сообщения, что игра завершена. В противном случае в бесконечном цикле `while(true)` { . . } генерируем случайное число в диапазоне от 0 до 8: `s = rand()*9/(RAND_MAX+1)`;

Мы учли, что функция `rand()` возвращает число от 0 до `RAND_MAX = 0x7fff`.

Проверяем, не занято ли окно с индексом `s`? Если не занято, присваиваем элементу массива `k[s]` значение 2 и перерисовываем это окно. Если же окно уже занято, то выполняем следующий шаг цикла, и так далее, пока не попадем на свободное окно. А такое окно обязательно существует, поскольку мы проверили, что занятых окон меньше 9.

Теперь осталось реализовать обработку сообщения `WM_PAINT`.

Начнем с определения индекса активного окна. Опять проверим, не является ли окно пустым? В этом случае ничего выводить не нужно, но для очистки окна лучше воспользоваться обработчиком сообщений по умолчанию `DefWindowProc()`, который закрасит окно фоновой кистью.

Если окно не является пустым, определим символ для вывода в окно. Для этого установим указатель `ch` на символ 'X', если `k[i] == 1`; или на символ 'O', если `k[i] == 2`. Мы специально ввели в массив `text` первый символ пробела, чтобы упростить выражение: `ch = text+k[i]`;

Определим дескриптор контекста устройства `BeginPaint()`, вычислим ограничивающий прямоугольник окна функцией `GetClientRect()` и выберем в качестве текущего шрифт `newFont`.

Функцией `DrawTextA()` выведем один символ в центр окна.

Для этого укажем флаги форматирования текста: `DT_SINGLELINE|DT_CENTER|DT_VCENTER`. Функция `EndPaint()` завершает обработку сообщения `WM_PAINT`.

Всплывающие окна

Всплывающие окна могут быть выведены в любом месте рабочего стола и всегда появляются на переднем плане, в отличие от дочерних, которые размещаются в клиентской области родительского окна. Рассмотрим технику создания всплывающих окон на примере приложения, строящего ху-график.

Обычно графические построения не производятся в главном окне программы, для этого создают отдельное окно. Поставим задачу: создать программу, которая читает текстовый файл с массивом числовых данных, расположенных в двух столбцах, и строит стандартный ху-график. В

качестве основы используем программу чтения текстового файла (см. листинг 2.3). Для упрощения кода перенесем описание вектора *v* и итератора *it* на глобальный уровень.

На глобальном уровне введем имя нового класса окна и опишем прототип оконной функции построения графика:

```
TCHAR WinClassGraphName[MAX_LOADSTRING] = _T("ChildClass");  
LRESULT CALLBACK WndGraph(HWND, UINT, WPARAM, LPARAM);
```

Для вывода графика добавим пункт меню **xy-график** с идентификатором ID_LINE. Обработчик этой команды разместим в функции главного окна:

```
case ID_LINE :  
    if (IsWindow(hGraph)) break;      RegisterGraphClass();  
    hGraph = CreateWindow(WinClassGraphName, _T("xy-график"), WS_SYSMENU |  
    WS_POPUP | WS_VISIBLE | WS_THICKFRAME | WS_CAPTION, sx/4, sy/4, min(sx, sy),  
    min(sx, sy), hWnd, 0, hInst, NULL); break;
```

Стили окна обеспечат вывод всплывающего окна WS_POPUP, ограниченного тонкой рамкой WS_THICKFRAME, с заголовком WS_CAPTION и системным меню WS_SYSMENU, отображаемое сразу после создания; стиль WS_VISIBLE. Дескриптор опишем в оконной функции главного окна: static HWND hGraph;

Для предотвращения повторного создания окна предусмотрим проверку его существования: if (IsWindow(hGraph)) break;

Положение всплывающего окна определяется в системе координат рабочего стола. Выберем такие параметры: sx/4, sy/4, min(sx, sy), min(sx, sy)

ПРИМЕЧАНИЕ

Для окна графика можно было бы выбрать стиль дочернего окна WS_CHILD. В этом случае, однако, необходим дополнительный стиль WS_EX_NOPARENTNOTIFY, чтобы предотвратить завершение приложения при закрытии дочернего окна.

Регистрацию *класса окна* выделим в отдельную функцию:

```
ATOM RegisterGraphClass()
```

```
{
```

```
WNDCLASSEX wcgraph      = {0};  
wcgraph.cbSize           = sizeof(WNDCLASSEX);  
wcgraph.style            = CS_HREDRAW |  
                          CS_VREDRAW;  
wcgraph.lpfnWndProc      = WndGraph;  
wcgraph.hInstance        = hInst;  
wcgraph.hCursor          = LoadCursor(NULL,  
                          IDC_CROSS);  
wcgraph.hbrBackground    = (HBRUSH)  
                          (COLOR_WINDOW+1);  
    wcgraph.lpszClassName = WinClassGraphName; wcgraph.hIconSm =  
    LoadIcon(hInst, MAKEINTRESOURCE(IDI_ICON1)); return RegisterClassEx(&wcgraph);  
}
```

ПРИМЕЧАНИЕ

Мы сделали присваивание wcgraph = {0}, чтобы не заполнять нулевые поля структуры WNDCLASSEX.

Для графического окна используем курсор IDC_CROSS и новую пиктограмму, которую мы импортировали в наш проект. Это можно сделать в контекстном меню **Add Resource...** окна **Resource View** (рис. 3.2).

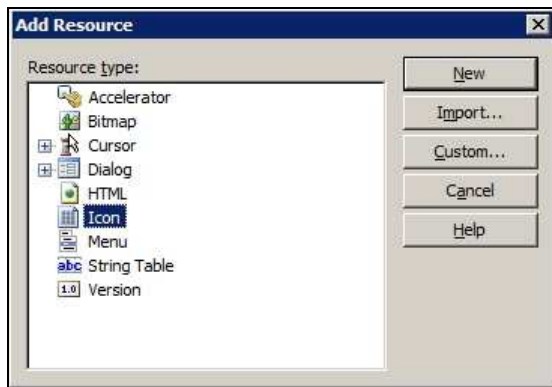


Рис. 3.2. Диалоговое окно **Add Resource**

По умолчанию новой иконке присваивается идентификатор `IDI_ICON1`. Рассмотрим оконную функцию всплывающего окна (листинг 3.2).

Листинг 3.2. Оконная функция построения ху-графика

```
const int scaleX = 8;           //Метки по оси x
const int scaleY = 4;           //Метки по оси y
const int indent = 25;          //Отступ для вывода меток оси
                                x

    struct DOUDLE_POINT { double x, y; }; const int
GRAPHSIZE = 1200; const int GRAPHWIDTH = 1000;
    LRESULT CALLBACK WndGraph(HWND hWnd, UINT message, WPARAM wParam, LPARAM
IParam)
    {
        PAINTSTRUCT ps; HDC hdc; static
HPEN hline; static HBRUSH hrect; RECT rt;
static int n, sx, sy, kx, ky; static double
max_x, max_y, min_x, min_y; int i, x, y;
        static POINT *pt; TCHAR s[20];
DOUDLE_POINT t; double z, hx, hy; static
DOUDLE_POINT *xy; switch (message) { case
WM_CREATE:
        if((n = v.size()) == 0)
        {
            MessageBox(hWnd, _T("Загрузите файл"), _T("Нет данных"),
MB_OK | MB_ICONHAND);
            DestroyWindow(hWnd); return 1;
        }
        pt = new POINT[n]; xy = new DOUDLE_POINT[n]; for (it =
v.begin(), i = 0; i < n; i++, it++)
        {
            if(sscanf(it->c_str(), "%lf %lf", &t.x, &t.y) != 2)
            {
                MessageBoxA(hWnd, it->c_str(), "Ошибка данных", MB_OK |
MB_ICONHAND);
                DestroyWindow(hWnd); return 1;
            }
            xy[i] = t;
        }
        max_x = min_x = xy[0].x; max_y = min_y = xy[0].y;
for (i = 1; i < n; i++)
        {
```



```

        if (max_x < xy[i].x) max_x = xy[i].x;    else if (min_x > xy[i].x) min_x =
xy[i].x;    if (max_y < xy[i].y) max_y = xy[i].y;    else if (min_y > xy[i].y) min_y =
xy[i].y;
    }
    hline = CreatePen(PS_SOLID, 6, RGB(0, 0, 255));    hrect =
CreateSolidBrush(RGB(255,0,0));
    hx = max_x - min_x;    hy = max_y - min_y;
    for (i = 0; i < n; i++)
    {
        pt[i].x = int((xy[i].x - min_x)*GRAPHWIDTH/hx + 0.5);    pt[i].y = int((xy[i].y -
min_y)*GRAPHWIDTH/hy + 0.5);
    }
    break; case WM_SIZE:
    sx = LOWORD(lParam);    sy =
HIWORD(lParam);    break; case WM_PAINT:
    hdc = BeginPaint(hWnd, &ps);    hx = (max_x -
min_x)/scaleX;    hy = (max_y - min_y)/scaleY;
    SetMapMode(hdc, MM_ANISOTROPIC);
    SetWindowExtEx(hdc, GRAPHSIZE, -GRAPHSIZE, NULL);
    SetViewportExtEx(hdc, sx, sy, NULL);
    SetViewportOrgEx(hdc, 2*indent, sy-indent, NULL);    SetTextAlign(hdc,
TA_RIGHT | TA_TOP);    for (z = min_x, i = 0; i <= scaleX; z += hx, i++)
    {
x = int((z - min_x)*GRAPHWIDTH/(max_x - min_x) + 0.5);
        _stprintf(s, _T("%.1lf"), z);
        TextOut(hdc, x, 0, s, _tcslen(s));
        MoveToEx(hdc, x, -10, NULL);
        LineTo(hdc, x, 10);
    }
    MoveToEx(hdc, 0, 0, NULL);
    LineTo(hdc, GRAPHWIDTH, 0);
    SetTextAlign(hdc, TA_RIGHT | TA_BOTTOM);    for (z = min_y, i = 0; i <=
scaleY; z += hy, i++)
    {
y = int((z - min_y)*GRAPHWIDTH/(max_y - min_y) + 0.5);
        _stprintf(s, _T("%.1lf"), z);
        TextOut(hdc, 0, y, s, _tcslen(s));
        MoveToEx(hdc, -10, y, NULL);
        LineTo(hdc, 10, y);
    }
    MoveToEx(hdc, 0, 0, NULL);
    LineTo(hdc, 0, GRAPHWIDTH);
    SelectObject(hdc, hline);
    Polyline(hdc, pt, n);    for (i = 0; i < n; i++)
    {
        SetRect(&rt, pt[i].x-8, pt[i].y-8, pt[i].x+8, pt[i].y+8);    FillRect(hdc, &rt, hrect);
    }
    EndPaint(hWnd, &ps);    break; case
WM_DESTROY:    DeleteObject(hline);
    DeleteObject(hrect);    delete[] pt;    delete[] xy;
    break;

```



```

default: return DefWindowProc(hWnd, message, wParam, lParam);
} return 0;
}

```

Создадим текстовый файл с данными и построим график, изображенный на рис. 3.3. Предполагается, что точки графика упорядочены по возрастанию координаты x .

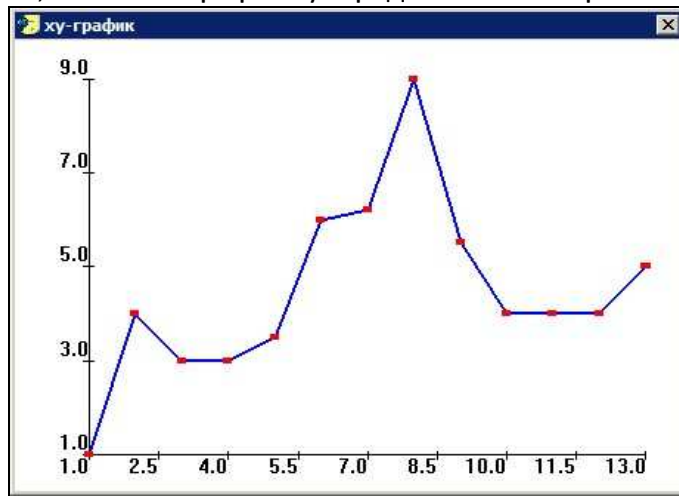


Рис. 3.3. Построение xy-графика

Константы, которые мы описали перед телом функции, служат для задания количества меток по осям координат, а также для выделения места для подписей осей. Мы выделили 25 логических единиц для подписи оси x и 50 единиц для оси y . Две константы `GRAPHSIZE` и `GRAPHWIDTH` определяют логические размеры окна и графика.

Начнем рассмотрение кода с сообщения `WM_CREATE`. Вначале проверим — загружены ли данные в память? `if((n = v.size()) == 0) . . .`

Если данные не загружены и n равно 0, выводим сообщение в диалоговом окне и закрываем окно `hGraph`.

ПРИМЕЧАНИЕ

Можно было бы решить эту проблему иначе. При создании окна сделать пункт меню построения графика **xy-graph** неактивным:

`EnableMenuItem(GetMenu(hWnd), ID_LINE, MF_DISABLED);` После загрузки файла вернуть активность пункту меню:

`EnableMenuItem(GetMenu(hWnd), ID_LINE, MF_ENABLED);`

Если же данные имеются, выделяем память под массив типа `DOUDLE_POINT`, где будем хранить данные графика в виде чисел с плавающей точкой типа `double`, и массив `POINT`, где координаты точек хранятся в логических единицах. Дело в том, что функция `Polyline()`, которую мы будем использовать для вывода графика, требует аргумента типа `POINT*`.

После чего извлекаем числовые данные из контейнера, где они хранятся в виде строк типа `string`.

Поскольку число строк текста нам известно, читаем их в цикле, и функцией `sscanf()` извлекаем по два числа из строки. Одна из причин, почему мы воспользовались этой функцией, заключается в том, что она возвращает количество преобразованных данных. Если же в строке оказалось менее двух чисел, выведем сообщение об ошибке в данных и закроем окно `hGraph`.

Здесь же определим диапазон для x и y -координат графика, т. е. минимальное и максимальное значения. Создадим перо для построения графика и кисть для выделения точек. После чего заполним массив логических координат точек графика.

При обработке сообщения `WM_SIZE` найдем размеры дочернего окна sx и sy (размер окна может быть изменен пользователем, и график должен быть перестроен).

Заранее предполагаем, что график будет строиться в логическом прямоугольнике размером 1000 1000. Размер окна определим 1200 1200, начало координат поместим в левом нижнем углу окна, выполнив смещение по оси x на 50, а по оси y на 25 единиц. Ось x направлена вправо, а ось y — вверх.

Формула преобразования:

$$x_{\text{лог}}^i = \left(\frac{x - x_{\text{imin}}}{x_{\text{max}} - x_{\text{min}}} \right) \cdot 1000; \quad y_{\text{лог}}^i = \left(\frac{y - y_{\text{min}}}{y_{\text{max}} - y_{\text{min}}} \right) \cdot 1000.$$

Для округления до целого добавим 0.5 и укажем явное преобразование типа.

Вывод графика осуществляется при обработке сообщения WM_PAINT.

Установим логическую систему координат 1200 1200 на всю клиентскую часть окна:

```
SetMapMode(hdc, MM_ANISOTROPIC);  
SetWindowExtEx(hdc, GRAPH_SIZE, -GRAPH_SIZE, NULL);  
SetViewportExtEx(hdc, sx, sy, NULL);
```

Ось x — вправо, а y — вверх. Начало координат определим относительно исходной системы координат (см. рис. 3.3).

```
SetViewportOrgEx(hdc, 2*indent, sy-indent, NULL);
```

Выведем подписи оси x, т. е. 8 числовых меток (как определено в константе scaleX), от минимального до максимального значения x-координаты.

При выводе горизонтальных меток хотелось бы, чтобы числовое значение своим правым краем стояло на позиции метки, но по умолчанию текст выравнивается влево. Решить эту проблему можно, установив выравнивание текста от правой верхней точки при помощи функции SetTextAlign().

Теперь весь вывод будет осуществляться, исходя из заданных параметров align: TA_RIGHT | TA_TOP.

Для преобразования числового значения в текстовый вид здесь так же можно воспользоваться функцией sprintf(). Зададим формат вывода "%.1lf", что означает 1 знак после десятичной точки для числа с плавающей точкой типа double (lf), размер поля вывода по фактической длине строки.

После вывода очередного числа ставим вертикальную риску длиной 20 логических единиц, а линию оси выведем после завершения цикла.

Подписи оси y сделаем аналогично, только для вывода числовых значений установим выравнивание по нижнему правому краю: TA_RIGHT | TA_BOTTOM. Выбираем синее перо hline и строим ломаную линию функцией Polyline(). После чего в цикле "заливаем" красной кистью hrect все точки квадратами со стороной в 16 единиц.

Нам осталось лишь освободить занятые ресурсы: при обработке сообщения WM_DESTROY — удалить перо, кисть и освободить память, занятую массивами pt и xu.

Диалоговые окна

Диалоговое окно является специальным типом окна, предназначенным для организации взаимодействия пользователя с программой. Диалоговое окно чаще всего используют как контейнер для элементов управления, оно предоставляет пользователю возможность выбирать или вводить данные. Начнем рассмотрение с *модальных диалоговых окон*. Модальное диалоговое окно при создании получает управление и не может вернуть его обратно вызывающему окну до завершения работы. Создается диалоговое окно при помощи вызова функции DialogBox():

```
INT_PTR WINAPI DialogBoxW (HINSTANCE hInstance, LPCWSTR lpTemplate, HWND  
hWndParent, DLGPROC lpDialogFunc);
```

- hInstance — дескриптор текущего приложения;
- lpTemplate — указатель ресурса диалогового окна;
- hWndParent — дескриптор "родительского" окна, породившего диалоговое окно;
- lpDialogFunc — указатель на функцию диалогового окна.

Диалоговое окно использует файл ресурсов, куда при помощи редактора ресурсов помещаются необходимые элементы.

ПРИМЕЧАНИЕ

На самом деле DialogBoxW() представляет собой макрос, который преобразуется в функцию DialogBoxParamW(), имеющую дополнительно параметр dwInitParam, равный 0 по умолчанию.

Для создания диалогового окна перейдем на подложку обозревателя ресурсов **Resource View** и в контекстном меню выберем **Insert Dialog** (рис. 3.4).

Окно мастера создания нового диалога показано на рис. 3.5, где по умолчанию размещены две стандартные кнопки **OK** и **Cancel**.

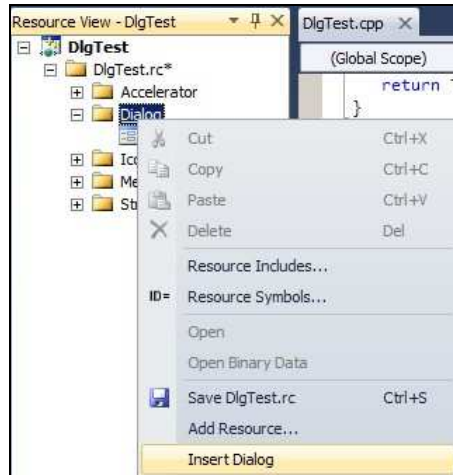


Рис. 3.4. Создание диалогового окна

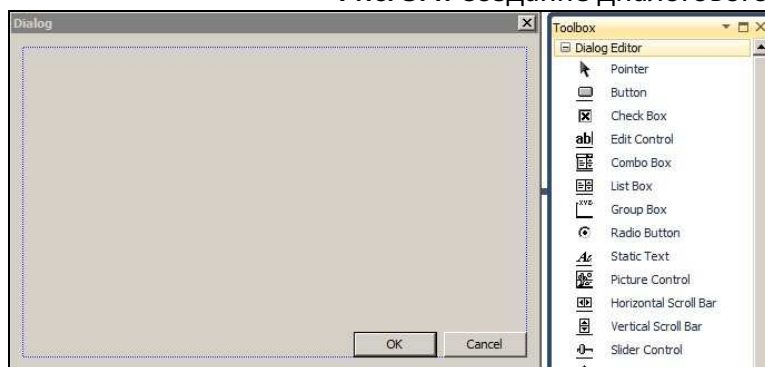


Рис. 3.5. Вид диалогового окна с панелью инструментов

Операционная система Windows имеет множество элементов управления, которые, по сути, являются окнами, зарегистрированными в системе. Это кнопки (**Button**), флажки/переключатели управления (**Check Box**), переключатели (**Radio Button**), списки (**List Box**), комбинированные списки с полем ввода (**Combo Box**), поля ввода

(**Edit Control**), полосы прокрутки (**Scroll Bar**), статические элементы (надписи) (**Static Text**) и др.

Элементы управления вставляются в диалоговое окно при помощи панели инструментов, хотя их можно вставить в приложение динамически, создавая окна элементов управления.

ПРИМЕЧАНИЕ

Каждый элемент управления имеет оконную функцию, принадлежащую операционной системе, которая используется совместно всеми приложениями. Управление элементами осуществляется посылкой их окну соответствующего сообщения. Данные передаются в параметрах сообщения, а результат возвращается функцией SendMessage().

Мы можем размещать элементы на поверхности диалогового окна буксировкой при нажатой левой кнопке мыши или выделением элемента на палитре и указанием прямоугольника для его размещения. После создания элемента управления можно изменить его размеры и другие свойства.

Рассмотрим пример создания диалогового окна, на котором разместим несколько наиболее часто используемых элементов управления.

Тестирование элементов управления

Создадим тестовую программу для следующих элементов управления: *кнопка, флажок, переключатель, список, комбинированный список (список с полем ввода), полоса прокрутки, статический элемент (надпись)*. В диалоговом окне мы будем изменять их состояние, а в главном окне выведем полученные значения. В качестве основы используем стандартную заготовку, а диалоговое окно, изображенное на рис. 3.6, построим в редакторе ресурсов. По умолчанию его идентификатор

IDD_DIALOG1.

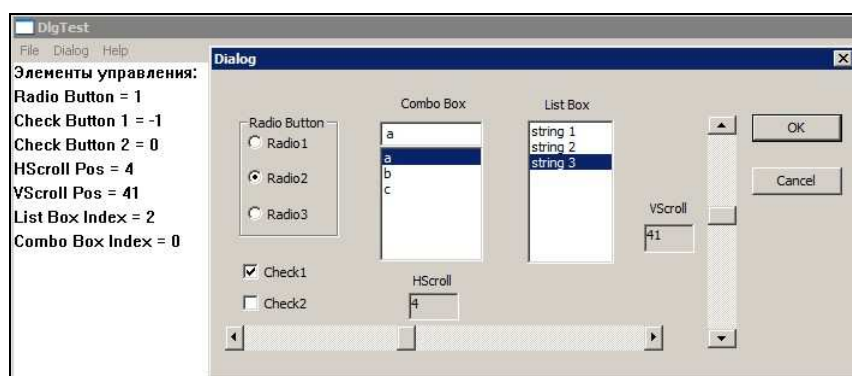


Рис. 3.6. Тест элементов управления

Каждому из элементов управления, размещенному на поверхности диалогового окна, за исключением статических полей, по умолчанию присваиваются уникальные идентификаторы, которые мы менять не будем. Заменим лишь идентификаторы статических полей, используемых для индикации состояния полос прокрутки (линеек скроллинга): IDC_HSCR, IDC_VSCR.

ПРИМЕЧАНИЕ

Для всех элементов управления установим свойство `Tabstop = true`, которое определит возможность перехода между элементами управления клавишей `<Tab>`. Исходный порядок соответствует последовательности размещения элементов на диалоговом окне. Для изменения порядка перехода воспользуйтесь меню **Format | Tab Order**. Щелкните мышью по элементам в необходимой последовательности и нажмите клавишу `<Enter>`.

Текст оконной функции приложения и функции диалогового окна, управляющей этими элементами, приведен в листинге 3.3.

Листинг 3.3. Тест элементов управления

```
INT_PTR CALLBACK Dialog1(HWND, UINT, WPARAM, LPARAM); static int radio, check1, check2, scrhl,
scrll, lIndex, cIndex; int *val[] = {&radio,&check1,&check2,&scrhl,&scrll,&lIndex,&cIndex};
TCHAR *combo[100] = { _T("a"), _T("b"), _T("c") };
TCHAR *list[100] = { _T("string 1"), _T("string 2"), _T("string 3") };
TCHAR *ctrl = _T("Элементы управления:");
TCHAR *str_control[] = { _T("Radio Button"),_T("Check Button 1"),
    _T("Check Button 2"),_T("HScroll Pos"),_T("VScroll Pos"),
    _T("List Box Index"),_T("Combo Box Index")};
const int HNUM = 10, VNUM = 100; const int List_size = 3, Combo_size = 3; const int INTERVAL = 20;
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    PAINTSTRUCT ps;
    HDC hdc;
    TCHAR str[256]; int i; switch (message)
    {
    case WM_COMMAND:
        switch (LOWORD(wParam))
```

```

{
case ID_STDDIALOG :
    DialogBox(hInst, MAKEINTRESOURCE(IDD_DIALOG1), hWnd, Dialog1);
        break;
        case IDM_EXIT: DestroyWindow(hWnd); break; default: return DefWindowProc(hWnd, message,
        wParam, lParam);
    } break;
case WM_PAINT:
    hdc = BeginPaint(hWnd, &ps);
    TextOut(hdc, 0, 0, ctrl, _tcslen(ctrl)); for (i = 0; i < 7;)
    {
        _stprintf(str, _T("%s = %d"), str_control[i], *val[i]);
        TextOut(hdc, 0, ++i*INTERVAL, str, _tcslen(str));
    }
    EndPaint(hWnd, &ps); break; case WM_DESTROY: PostQuitMessage(0); break;
default: return DefWindowProc(hWnd, message, wParam, lParam);
}
return 0;
}
////////////////////////////////////
INT_PTR CALLBACK Dialog1(HWND hDlg, UINT message, WPARAM wParam, LPARAM lParam)
{
    static int radio, check1, check2, scrLh, scrLv; static HWND hScroll, vScroll, hWndList, hWndComboBox;
    int i; switch (message)
    {
        case WM_INITDIALOG: radio = ::radio;
            CheckRadioButton(hDlg, IDC_RADIO1, IDC_RADIO3, IDC_RADIO1+radio); check1 = ::check1;
            SendDlgItemMessage(hDlg, IDC_CHECK1, BM_SETCHECK, check1, 0); check2 = ::check2;
            SendDlgItemMessage(hDlg, IDC_CHECK2, BM_SETCHECK, check2, 0); scrLh = ::scrLh;
            SetDlgItemInt(hDlg, IDC_HSCR, scrLh, 0); hScroll = GetDlgItem(hDlg, IDC_SCROLLBAR1);
            SetScrollRange(hScroll, SB_CTL, 0, HNUM, FALSE); SetScrollPos(hScroll, SB_CTL, scrLh, TRUE); scrLv =
            ::scrLv;
            SetDlgItemInt(hDlg, IDC_VSCR, scrLv, 0); vScroll = GetDlgItem(hDlg, IDC_SCROLLBAR2);
            SetScrollRange(vScroll, SB_CTL, 0, VNUM, FALSE); SetScrollPos(vScroll, SB_CTL, scrLv, TRUE);
            hWndList = GetDlgItem(hDlg, IDC_LIST1);
            for (i = 0; i < List_size; i++)
                SendMessage(hWndList, LB_ADDSTRING, 0, (LPARAM)list[i]); SendMessage(hWndList,
            LB_SETCURSEL, lIndex, 0); hWndComboBox = GetDlgItem(hDlg, IDC_COMBO1); for (i = 0; i <
            Combo_size; i++)
                SendMessage(hWndComboBox, CB_ADDSTRING, 0, (LPARAM)combo[i]);
            SendMessage(hWndComboBox, CB_SETCURSEL, cIndex, 0); return TRUE; case WM_COMMAND:
            switch(LOWORD(wParam))
            {
                case IDOK: lIndex = SendMessage(hWndList, LB_GETCURSEL, 0, 0); cIndex =
                SendMessage(hWndComboBox, CB_GETCURSEL, 0, 0);
                ::radio = radio;
                ::check1 = check1;
                ::check2 = check2;
                ::scrLh = scrLh;
                ::scrLv = scrLv;
            }
    }
}

```

```

    InvalidateRect(GetParent(hDlg), NULL, 1); case IDCANCEL : return EndDialog(hDlg, 0); case
IDC_CHECK1 : check1 = ~check1;
    SendDlgItemMessage(hDlg, IDC_CHECK1, BM_SETCHECK, check1, 0); return TRUE;
case IDC_CHECK2 : check2 = ~check2;
    SendDlgItemMessage(hDlg, IDC_CHECK2, BM_SETCHECK, check2, 0); return TRUE;
case IDC_RADIO1 : radio = 0; break; case IDC_RADIO2 : radio = 1; break; case IDC_RADIO3 : radio = 2;
break;
}
    CheckRadioButton(hDlg, IDC_RADIO1, IDC_RADIO3, IDC_RADIO1+radio); return TRUE; case
WM_HSCROLL:
    switch(LOWORD(wParam))
    {
        case SB_LINELEFT : scrLh--; break; case SB_LINERIGHT : scrLh++; break; case SB_PAGELEFT :
scrLh -= HNUM/2; break; case SB_PAGERIGHT : scrLh += HNUM/2; break;
        case SB_THUMBPOSITION : scrLh = HIWORD(wParam); break;
    }
    scrLh = max(0, min(scrLh, HNUM)); if (scrLh != GetScrollPos(hScroll, SB_CTL))
    {
        SetScrollPos(hScroll, SB_CTL, scrLh, TRUE);
        SetDlgItemInt(hDlg, IDC_HSCR, scrLh, 0);
    }
    return TRUE; case WM_VSCROLL:
    switch(LOWORD(wParam))
    {
        case SB_LINEUP : scrLv--; break; case SB_LINEDOWN : scrLv++; break; case SB_PAGEUP : scrLv -=
VNUM/10; break; case SB_PAGEDOWN : scrLv += VNUM/10; break; case SB_THUMBPOSITION : scrLv
= HIWORD(wParam); break;
    }
    scrLv = max(0, min(scrLv, VNUM)); if (scrLv != GetScrollPos(vScroll, SB_CTL))
    {
        SetScrollPos(vScroll, SB_CTL, scrLv, TRUE);
        SetDlgItemInt(hDlg, IDC_VSCR, scrLv, 0);
    }
    return TRUE;
default: return FALSE;
}
return FALSE;
}

```

На глобальном уровне опишем переменные, которые будут отражать состояние элементов управления: static int radio, check1, check2, scrLh, scrLv, lIndex, cIndex;

Поскольку принято, что в группе переключателей (Radio Button) может выбираться лишь один переключатель, то для описания всей группы достаточно одной переменной radio целого типа, значения которой {0,1,2} мы понимаем как включение первой, второй или третьей кнопки соответственно.

ПРИМЕЧАНИЕ

Вообще-то нет никаких проблем для включения нескольких переключателей одновременно при помощи функции CheckDlgButton() или же просто посылкой сообщения BST_CHECKED для включения и BST_UNCHECKED для выключения, однако лучше придерживаться стандартных соглашений.

Флажки (Check Box) могут переключаться независимо друг от друга, поэтому для их описания определим две переменные check1 и check2. Признаком включения является *ненулевое* значение переменной.

Переменные scrLh и scrLv отражают позицию горизонтальной и вертикальной полосы прокрутки Scroll Bar.

ПРИМЕЧАНИЕ

Здесь полосы прокрутки являются элементами управления, в отличие от рассмотренных ранее стилей, и могут располагаться в любом месте окна.

И, наконец, переменные lIndex, cIndex необходимы для сохранения индекса выбранного элемента списка и комбинированного списка.

Эти переменные мы объявили как static лишь для того, чтобы они имели нулевое начальное значение.

Для начального заполнения списков мы описали два массива указателей list[100] и combo[100] типа TCHAR*, которым присвоили лишь 3 первых значения.

В функции главного окна WndProc() необходимо добавить обработчик пункта меню **StdDialog** с идентификатором ID_STDDIALOG: DialogBox(hInst, MAKEINTRESOURCE(IDD_DIALOG1), hWnd, Dialog1); Здесь будет сделан вызов диалогового окна IDD_DIALOG1.

Вывод значений параметров сделаем в сообщении WM_PAINT главного окна, где выведем в цикле значение переменных, отслеживающих состояние элементов управления.

Для преобразования числового значения в строку проще всего воспользоваться функцией форматного вывода _stprintf(). Строка формата позволит "склеить" текст и числовое значение: "%s=%d". Выводим строки с интервалом в 20 логических единиц.

Вся логика обработки осуществляется в функции диалогового окна Dialog1(), рассмотрим подробнее ее работу.

Грамотно оформленное диалоговое окно должно иметь возможность закрыться как кнопкой **ОК**, так и **Cancel**, поэтому нам нужен второй комплект локальных переменных для описания состояния элементов управления. static int radio, check1, check2, scrLh, scrLv;

Для лучшей читаемости кода мы сохранили те же имена, помня, что локальная переменная всегда имеет приоритет перед глобальной. Для обращения к глобальной переменной используем явное указание области видимости "::".

При открытии диалогового окна всегда генерируется сообщение WM_INITDIALOG, где мы и присвоим начальное значение переменных, например: radio = ::radio; и произведем начальную установку элементов управления:

Переключатели

Устанавливаем выбранную кнопку функцией CheckRadioButton():

```
BOOL WINAPI CheckRadioButton(HWND hDlg, int nIDFirstButton, int nIDLastButton,
int nIDCheckButton);
```

Функция принимает дескриптор диалогового окна hDlg, которое является родительским для всех элементов управления, диапазон идентификаторов переключателей (IDC_RADIO1, IDC_RADIO3), и идентификатор активной кнопки IDC_RADIO1+radio. Для корректной работы данной функции необходимо создавать кнопки непрерывно одну за другой, чтобы их идентификаторы, которые эквивалентны целым числам, представляли собой последовательный ряд чисел. Должно быть понятно, что при значении переменной radio: 0,1,2; выражение IDC_RADIO1+radio будет эквивалентно одному из трех идентификаторов:

IDC_RADIO1, IDC_RADIO2, IDC_RADIO3.

ПРИМЕЧАНИЕ

Вставим селекторные кнопки в рамку элемента управления Group Box, который будет выполнять чисто декоративную функцию.

Флажки

Установим флажки, передавая им сообщение BM_SETCHECK функцией `SendDlgItemMessage()`:

```
LRESULT WINAPI SendDlgItemMessageW(HWND hDlg, int nIDDlgItem,
UINT Msg, WPARAM wParam, LPARAM lParam);
```

Функция принимает дескриптор окна диалога `hDlg`, идентификатор кнопки и сообщение `BM_SETCHECK`. Отличное от нуля значение `wParam` означает установку флажка кнопки, а нулевое значение снимает флажок. `lParam` здесь не используется.

Полоса прокрутки

Для отображения состояния горизонтальной и вертикальной полосы прокрутки мы вставили два статических элемента с идентификаторами `IDC_HSCR` и `IDC_VSCR`. Отобразим в них значение переменных `scrh` и `scriv` функцией `SetDlgItemInt()`:

```
BOOL WINAPI SetDlgItemInt(HWND hDlg, int nIDDlgItem, UINT uValue,
BOOL bSigned);
```

Функция принимает: `hDlg` — дескриптор окна, `nIDDlgItem` — идентификатор элемента управления, `uValue` — переменную целого типа, `bSigned` — признак вывода числа со знаком.

Нужно получить дескриптор полосы прокрутки функцией `GetDlgItem()`:

```
HWND WINAPI GetDlgItem(HWND hDlg, int nIDDlgItem);
```

Это специализированная функция для диалогового окна, которая позволяет по идентификатору элемента управления `nIDDlgItem` получить его дескриптор.

Теперь нужно задать диапазон для горизонтальной полосы прокрутки. Это делается при помощи функции `SetScrollRange()`:

```
BOOL WINAPI SetScrollRange(HWND hWnd, int nBar, int nMinPos, int nMaxPos, BOOL
bRedraw);
```

Первым параметром указываем дескриптор полосы прокрутки. Второй параметр имеет значение `SB_CTL`, что идентифицирует полосу прокрутки как элемент управления, а не свойство окна, где он принимал значение `SB_HORZ` и `SB_VERT`. Следующие два параметра задают `min` и `max` значение позиции полос прокрутки (мы установим диапазон `[0, HNUM]`, переменная `HNUM` описана в заголовке программы), а последний параметр служит признаком перерисовки. Затем задается текущая позиция движка функцией `SetScrollPos()`: `int WINAPI SetScrollPos(HWND hWnd, int nBar, int nPos, BOOL bRedraw);` Первые два параметра имеют тот же смысл, что и в предыдущей функции, 3-й параметр — позиция движка, а последний параметр — признак перерисовки.

Параметры вертикальной полосы прокрутки устанавливаем аналогично.

Список

Для первичного заполнения списка мы описали на глобальном уровне массив указателей `TCHAR *list[]`, где значение присвоено только 3-м первым элементам.

Получим дескриптор списка `hWndList` и в цикле заполним список, передавая сообщение `LB_ADDSTRING`, где указатель на строку помещаем в `lParam` с явным преобразованием типа, иначе компилятор не сможет построить код этого выражения. Константа `List_size` описана на глобальном уровне и равна 3.

Для отображения выделенного элемента посылаем списку сообщение `LB_SETCURSEL`, передавая в `wParam` значение индекса `lIndex`.

Комбинированный список

Опишем на глобальном уровне массив указателей `TCHAR *combo[]` для начальной инициализации комбинированного списка.

Далее все делается так же, как у списка, только префикс сообщений вместо `LB` будет `CB`.

ПРИМЕЧАНИЕ

Если для списка или комбинированного списка установлен стиль Sort, отображаться элементы будут в отсортированном по умолчанию порядке.

Статические элементы (надписи)

Обычно статические элементы используют для вывода в окне поясняющих надписей, но в этом случае к ним нет необходимости обращаться, и мы оставим идентификаторы со значением по умолчанию IDC_STATIC. Однако двум элементам, которые используются для вывода состояния полос прокрутки, присвоим уникальные идентификаторы IDC_HSCR и IDC_VSCR. Чтобы выделить эти элементы в окне установим true для стиля Border и Client Edge.

Выводят в окно статический элемент обычно функциями: SetDlgItemText() или SetDlgItemInt(), передавая им в качестве параметра указатель на строку текста или переменную целого типа.

При совершении операций с элементами управления в диалоговую функцию передается сообщение WM_COMMAND, а в LOWORD(wParam) содержится идентификатор этого элемента. Поэтому для обработки сообщений от элементов управления внутри обработчика сообщения WM_COMMAND обычно создают переключатель switch, в котором и выполняют селекцию поступающих сообщений. Причем, если оконная функция диалогового окна обрабатывает сообщение, она должна возвращать TRUE.

```
case WM_COMMAND:
    switch(LOWORD(wParam))
    {
        ...
    }
    return TRUE;
```

Начнем рассмотрение с кнопок **OK** и **Cancel**. Мастер построения диалогового окна автоматически вставляет эти кнопки, присваивая им стандартные идентификаторы IDOK и IDCANCEL.

ПРИМЕЧАНИЕ

Идентификатор IDCANCEL имеет значение, равное 2, такое же сообщение генерируется при нажатии системной кнопки закрытия окна.

При нажатии кнопки **OK** необходимо сохранить на глобальном уровне значение локальных переменных, контролирующих элементы управления, например:

```
::radio = radio;
```

а индексы элементов, выбранных в списке и комбинированном списке, необходимо получить, послав этим элементам сообщения LB_GETCURSEL и CB_GETCURSEL соответственно. Возвращаемое значение функции SendMessage() вернет искомым индекс. Далее необходимо инициировать перерисовку главного окна вызовом функции InvalidateRect(), где дескриптор родительского окна мы получим как возвращаемое значение функции GetParent():

```
HWND WINAPI GetParent(HWND hWnd);
```

ПРИМЕЧАНИЕ

Мы намеренно не использовали оператор break в конце обработчика сообщения от кнопки **OK**. В этом случае будет выполняться код следующего оператора case и произойдет нормальное закрытие диалогового окна.

При нажатии кнопки **Cancel** выполнение кода фрагмента начнется с функции EndDialog():

```
BOOL WINAPI EndDialog(HWND hDlg, int nResult);
```

здесь мы не сохраняем локальные переменные и не перерисовываем окно.

При обработке сообщений от флажка IDC_CHECK1 и IDC_CHECK2 инвертируем значение переменной. Поскольку ее начальное значение 0, то ~0 равно -1, а ~(-1) равно 0. Таким образом, переменная будет принимать одно из двух значений 0 или -1, что интерпретируется как состояние флажка "**выключено**" (сброшен) и "**включено**" (установлен).

Для изменения состояния флажка посылаем ему сообщение `BM_SETCHECK`. Нулевое значение `wParam` сбросит, а ненулевое — установит флажок.

А вот переключатели (селекторные кнопки) нужно обрабатывать вместе, поскольку во включенном состоянии может быть только один. Мы поместили обработку сообщений от этих переключателей в конец блока, чтобы иметь возможность после установки значения переменной `radio` передать управление функции `CheckRadioButton()` для изменения их состояния.

Обработка сообщений от полос прокрутки принципиально отличается от прочих элементов управления тем, что они не генерируют сообщения `WM_COMMAND`, а точно так же, как и полосы прокрутки — свойства окна, генерируют сообщения: `WM_HSCROLL` — для горизонтальной и `WM_VSCROLL` — для вертикальной полосы прокрутки. Обработка этих сообщений будет осуществляться точно так же, как и в листинге 2.3, однако имеется одно принципиальное отличие — в качестве идентификатора полосы прокрутки — элемента управления — используется `SB_CTL`.

Мы будем обрабатывать сообщения от полосы прокрутки: `SB_LINELEFT`, `SB_LINERIGHT`, `SB_LINEUP`, `SB_LINEDOWN`, `SB_PAGELEFT`, `SB_PAGERIGHT`, `SB_PAGELF`, `SB_PAGEDOWN`, `SB_THUMBPOSITION`.

Так, для горизонтальной полосы прокрутки мы изменяем значение переменной `scrH`, причем, поскольку мы выбрали для `HNUM` значение 10, то при щелчке на полосе прокрутки сделаем шаг передвижения равным 5 (`HNUM/2`).

После установки нового значения переменной, как и ранее, при помощи стандартной конструкции, позаботимся о том, чтобы не выйти за пределы интервала `[0,HNUM]`. После чего отобразим значение переменной `scrH` в статическом элементе (надписи) `IDC_HSCR` функцией `SetDlgItemInt()`.

Вертикальный скроллинг организуем аналогично, имея в виду, что переменная `scrV`, которая его контролирует, изменяется на отрезке `[0,VNUM]`, а шаг здесь равен 10 (`VNUM/10`). Для отображения состояния вертикального скроллинга используется статический элемент с идентификатором `IDC_VSCR`.

Итак, все сообщения от элементов управления обработаны и возвращают `TRUE`, все же остальные сообщения игнорируются и возвращают `FALSE`.

После закрытия диалогового окна главное окно перерисовывается для вывода значений переменных, контролирующих состояние элементов управления. Поскольку мы определили переменные на глобальном уровне, то они доступны и в оконной функции главного окна. Здесь же опишем массив строк с поясняющим текстом.

Чтобы не писать код для вывода каждой переменной, опишем массив указателей и присвоим его элементам адреса этих переменных: `int *val[] = {&radio,&check1,&check2,&scrH,&scrV,&IIndex,&CIndex};`

Теперь при обработке сообщения `WM_PAINT`, после вывода заголовка `ctrl` можно организовать цикл для вывода значений переменных. При формировании строки вывода мы воспользовались функцией форматного вывода `_sprintf()`. Выражение `++i*INTERVAL` обеспечит увеличение индекса и наращивание *y*-координаты на 20 единиц.

Общие элементы управления

Элементы управления, которые появились лишь при создании ОС Windows 95, получили название *общих элементов управления* (*Common Controls*). Они расширяют возможности стандартных элементов управления и придают программам современный вид. Один из этих элементов мы уже рассмотрели в главе 2 — это панель инструментов (**Tool Bar**). Сейчас рассмотрим еще 3 элемента — **наборный счетчик (Spin Control)**, **ползунковый регулятор (Slider Control)** и **индикатор выполнения (Progress Bar Control)**.

Техника их использования мало чем отличается от стандартных элементов управления: в редакторе ресурсов перемещаем нужные элементы с палитры инструментов в диалоговое окно, присваиваем идентификаторы и устанавливаем необходимые свойства.

Особенностью же общих элементов управления является то, что они не входят в стандартную библиотеку и для них необходимо добавить файл включений `commctrl.h`, а также подключить к проекту библиотеку общих элементов управления `comctl32.lib`.

К тому же, до использования общих элементов управления приложение должно инициализировать библиотеку функцией `InitCommonControls()`.

ПРИМЕЧАНИЕ

Хотя панель инструментов и является общим элементом управления, для ее использования нет необходимости в вызове функции `InitCommonControls()`.

Так же, как и стандартные элементы управления, общие элементы являются специальными элементами, которые посылают родительскому окну сообщение `WM_COMMAND` или `WM_NOTIFY`. Управление осуществляется посылкой им соответствующих сообщений. Демонстрационная задача приведена на рис. 3.7 и далее в листинге 3.4.

Создадим диалоговое окно, которое вызывается в пункте меню **StdDialog**, и разместим там элементы управления: наборный счетчик (спин), ползунковый регулятор (ползунок), индикатор выполнения и два статических элемента для вывода значений спина и ползунка. При закрытии окна кнопкой **ОК** возвращаем установленные значения элементов управления и выводим в окне, при нажатии на кнопку **Cancel** ничего возвращать не будем.

ПРИМЕЧАНИЕ

Все рассмотренные общие элементы управления (`Spin Control`, `Slider Control`, `Progress Bar Control`) имеют внутреннюю память для хранения состояния элемента.

Для обмена данными опишем на глобальном уровне три переменные: `spin`, `track`, `progress`. Вывод в окно осуществим в сообщении `WM_PAINT`, где сформируем строку вывода функцией `format` в строку `_sprintf()`. Причем используем универсальную функцию, работающую как с C-строкой, так и со строкой `Unicode`. Для вывода воспользуемся функцией `DrawText()`, поскольку она позволяет форматировать выводимый текст. Так что мы сможем одним оператором вывести три строки текста, выравнивая данные табуляцией.

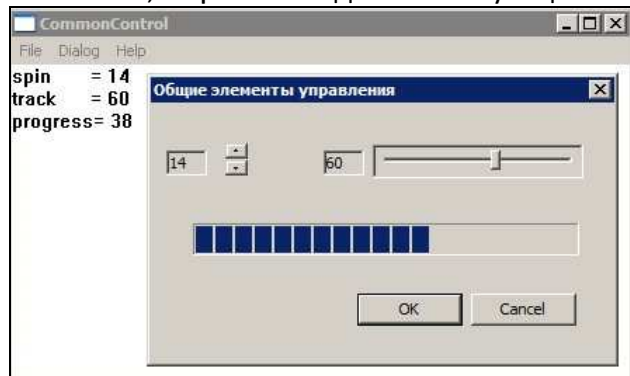


Рис. 3.7. Диалоговое окно с общими элементами управления

Slider (Track Bar Control) — ползунок

Ползунок или ползунковый регулятор состоит из указателя, движущегося вдоль шкалы. Он является разновидностью горизонтальной полосы прокрутки и генерирует сообщение `WM_HSCROLL`. В `IParam` передается дескриптор элемента управления, породившего сообщение, и мы можем решить вопрос о том, каким элементом управления оно вызвано, если таких элементов несколько.

В функции диалогового окна опишем переменную для хранения состояния ползунка и его дескриптор:

```
static int track; static HWND hTrack;
```

В сообщении об инициализации диалога необходимо определить дескриптор, минимальную и максимальную позиции ползунка и установить начальное значение:

```
track = ::track;  
hTrack = GetDlgItem(hDlg, IDC_SLIDER1); SendMessage(hTrack, TBM_SETRANGEMIN, 0, 0);  
SendMessage(hTrack, TBM_SETRANGEMAX, 0, 100);
```

`SendMessage(hTrack, TBM_SETPOS, TRUE, track);`

Здесь минимальное и максимальное значения позиции ползунка определяются посылкой ему сообщения TBM_SETRANGEMIN и TBM_SETRANGEMAX, а само значение передается в lParam, у нас — [0,100]. Текущая позиция задается посылкой сообщения TBM_SETPOS, где значение TRUE в wParam означает необходимость перерисовки ползунка, а позиция track передается в lParam.

ПРИМЕЧАНИЕ

Вместо двух сообщений для установки минимального и максимального значений ползунка можно использовать сообщение TBM_SETRANGE, где минимум задается в младшем слове lParam, а максимум — в старшем.

`SendMessage(hTrack, TBM_SETRANGE, 0, 100<<16);`

Теперь осталось вывести значение track в статическом элементе IDC_TR1.

`SetDlgItemInt(hDlg, IDC_TR1, track, 0);`

Для манипуляций с ползунком в сообщении WM_HSCROLL передадим ему сообщение TBM_GETPOS. Позицию ползунка получим из младшего слова возвращаемого значения функции SendMessage(). После чего выведем это значение в статическом элементе IDC_TR1 так же, как мы это делали ранее при инициализации ползунка.

ПРИМЕЧАНИЕ

Когда ползунок находится в фокусе ввода (элемент управления, находящийся в фокусе, готов принимать команды с клавиатуры), его состоянием можно управлять клавишами управления курсором: <End> — минимальное значение, <Home> — максимальное значение, < > и < > — увеличение значения на 1, < > и < > — уменьшение значения на 1, <Page Up> — увеличение значения на шаг, <Page Down> — уменьшение значения на шаг. Шаг ползунка определяется автоматически в 1/5 от диапазона его изменения.

Spin (Up-Down Control) — наборный счетчик

Наборный счетчик представляет собой специальный вид полосы вертикального скроллинга, он состоит из кнопок вверх и вниз так же, как и элемент управления **Vertical Scroll Bar**, и порождает сообщение WM_VSCROLL. В lParam передается дескриптор элемента управления, породившего сообщение.

В оконной функции диалогового окна необходимо описать дескриптор спина hSpin, а в сообщении об инициализации диалога определить этот дескриптор и установить его начальное значение.

Опишем в функции диалогового окна дескриптор спина: static HWND

hSpin;

Для отображения значения спина в статическом элементе можно поступить как для ползунка — вывести значение функцией SetDlgItemInt(). Однако счетчик предоставляет пользователю дополнительный сервис. Если при построении диалогового окна установить свойство спина **Set Buddy Integer**, то можно этому спину назначить "приятельское" (buddy) окно IDC_SP1, а, посылая ему сообщение UDM_SETBUDDY, в wParam передать дескриптор "приятеля" hBuddy:

`hSpin = GetDlgItem(hDlg, IDC_SPIN1); hBuddy =`

`GetDlgItem(hDlg, IDC_SP1);`

`SendMessage(hSpin, UDM_SETBUDDY, (WPARAM)hBuddy, 0);`

В этом случае счетчик сам позаботится о выводе своего значения в приятельское окно, и нам об этом беспокоиться больше не нужно.

Сообщение UDM_SETRANGE устанавливает диапазон изменения счетчика так же, как для ползунка.

`SendDlgItemMessage(hDlg, IDC_SPIN1, UDM_SETRANGE, 0, 100);`

ПРИМЕЧАНИЕ

Если в диапазоне значений спина минимальное значение превышает максимальное, LOWORD(lParam) > HIWORD(lParam), инвертируется направление изменения спина.

Сообщение UDM_SETPOS устанавливает позицию спина, передаваемую в lParam.


```
SendMessage(hSpin, UDM_SETPOS, 0, spin);
```

Поскольку мы установили для наборного счетчика (спина) приятельское окно, нет необходимости в обработке сообщения WM_VSCROLL для индикации числового значения.

Progress Bar Control — индикатор выполнения

Индикатор выполнения — это элемент управления, который показывает течение процесса, например, при копировании файла в файловом менеджере. Управление осуществляется передачей сообщений, сам же индикатор сообщений не генерирует.

Как и для предыдущих элементов управления, опишем в диалоговой оконной функции дескриптор индикатора: static HWND hProgress; и определим его при инициализации диалогового окна. hProgress = GetDlgItem(hDlg, IDC_PROGRESS1);

Для индикатора необходимо определить диапазон изменения и задать начальную позицию. Диапазон [0,100] определим посылкой сообщения PBM_SETRANGE, где минимум задается в младшем слове lParam, а максимум — в старшем. Шаг индикатора, равный 1, зададим в wParam сообщения PBM_SETSTEP. Начальную позицию, равную t, зададим посылкой сообщения PBM_SETPOS. Здесь t — статическая переменная целого типа, описанная в оконной функции диалога, будет контролировать состояние индикатора.

```
SendMessage(hProgress, PBM_SETRANGE, 0, 100<<16);
```

```
SendMessage(hProgress, PBM_SETSTEP, 1, 0);
```

```
SendMessage(hProgress, PBM_SETPOS, t, 0);
```

Для создания иллюзии движения создадим таймер с интервалом в 0,1 секунды:

```
SetTimer(hDlg, 1, 100, NULL);
```

Теперь в обработчике сообщения от таймера WM_TIMER будем увеличивать значение переменной на 1 и переустанавливать состояние индикатора.

```
if (++t > 99) t = 0;
```

```
SendMessage(hProgress, PBM_SETPOS, t, 0);
```

Оператором if через каждые 10 секунд сбрасываем индикатор в начальное состояние.

Осталось только перед закрытием диалогового окна кнопкой **ОК** передать глобальной переменной значение индикатора и уничтожить таймер:

```
progress = t;
```

```
KillTimer(hDlg, 1);
```

Поскольку состояние ползунка отслеживает переменная track, передадим ее на глобальный уровень:

```
::track = track;
```

Считываем состояние наборного счетчика, передавая ему сообщение UDM_GETPOS, и перерисовываем главное окно функцией InvalidateRect() (см. листинг 3.4). После чего диалоговое окно можно закрыть.

При нажатии кнопки **Cancel** уничтожаем таймер и закрываем диалоговое окно без перерисовки главного окна.

Листинг 3.4. Тест общих элементов управления

```
#include <commctrl.h>
INT_PTR CALLBACK Dialog1(HWND, UINT, WPARAM, LPARAM); static int spin,
track, progress;
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM
lParam)
{
    PAINTSTRUCT ps;
    HDC hdc;
    TCHAR str[256]; RECT rt; switch
(message)
{
```

```

        case WM_CREATE:
InitCommonControls(); break; case
WM_COMMAND:
    switch (LOWORD(wParam))
    {
        case ID_COMMCTRL:
            DialogBox(hInst, MAKEINTRESOURCE(IDD_DIALOG1), hWnd, Dialog1); break;
        case IDM_EXIT: DestroyWindow(hWnd); break; default: return
DefWindowProc(hWnd, message, wParam, lParam);
    }
    break; case WM_PAINT:
        SetRect(&rt, 0, 0, 100, 100); hdc = BeginPaint(hWnd,
&ps); _sprintf(str, _T("spin\t= %d\ntrack\t= %d\nprogress=
%d"), spin, track, progress);
        DrawText(hdc, str, _tcslen(str), &rt, DT_LEFT | DT_EXPANDTABS); EndPaint(hWnd,
&ps); break;
        case WM_DESTROY: PostQuitMessage(0); break; default: return
DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}
////////////////////////////////////
INT_PTR CALLBACK Dialog1(HWND hDlg, UINT message, WPARAM wParam, LPARAM lParam)
{ static int t, track; static HWND hSpin, hBuddy, hTrack,
hProgress; switch (message)
{
    case WM_INITDIALOG: track = ::track;
        SetDlgItemInt(hDlg, IDC_TR1, track, 0); hTrack = GetDlgItem(hDlg,
IDC_SLIDER1); SendMessage(hTrack, TBM_SETRANGE, 0, 100<<16);
SendMessage(hTrack, TBM_SETPOS, TRUE, track); hSpin = GetDlgItem(hDlg,
IDC_SPIN1); hBuddy = GetDlgItem(hDlg, IDC_SP1);
        SendMessage(hSpin, UDM_SETBUDDY, (WPARAM)hBuddy, 0);
        SendMessage(hSpin, UDM_SETRANGE, 0, 100); SendMessage(hSpin,
UDM_SETPOS, 0, spin); hProgress = GetDlgItem(hDlg, IDC_PROGRESS1);
SendMessage(hProgress, PBM_SETRANGE, 0, 100<<16);
        SendMessage(hProgress, PBM_SETSTEP, 1, 0);
        SendMessage(hProgress, PBM_SETPOS, t, 0); SetTimer(hDlg, 1,
100, NULL); return TRUE; case WM_TIMER :
        if (++t > 99) t = 0;
        SendMessage(hProgress, PBM_SETPOS, t, 0); return TRUE; case
WM_HSCROLL:
        track = LOWORD(SendMessage(hTrack, TBM_GETPOS, 0, 0)); SetDlgItemInt(hDlg, IDC_TR1,
track, 0);
        return TRUE;
        case WM_COMMAND:
            switch(LOWORD(wParam))
            {
                case IDOK : progress = t; ::track = track;
                    spin = SendMessage(hSpin, UDM_GETPOS, 0, 0);
                    InvalidateRect(GetParent(hDlg), NULL, 1); case IDCANCEL:

```



```

        KillTimer(hDlg,1); EndDialog(hDlg, 0);
return TRUE; default: return FALSE;
    }
    default: return FALSE;
}
return FALSE;
}

```

Окно редактирования

Для ввода данных чаще всего используется элемент управления — *окно редактирования* (Edit Box Control). Управление окном осуществляется передачей ему сообщений или специализированными функциями диалогового окна:

- ❑ GetDlgItemText(hDlg, IDC_EDIT, text, length) — возвращает в TCHAR массив text не более length символов окна редактирования;
- ❑ SetDlgItemText(hDlg, IDC_EDIT, text) — заполняет окно редактирования содержимым TCHAR-строки text;
- ❑ GetDlgItemInt(hDlg, IDC_EDIT, lpTranslated, bSigned) — функция возвращает целое число, в которое преобразуется содержимое окна редактирования. lpTranslated — указатель переменной типа BOOL, которая устанавливается в TRUE при успешном преобразовании числа, и FALSE — в противном случае. Если bSigned равно TRUE, преобразуется число со знаком иначе, число рассматривается как беззнаковое;
- ❑ SetDlgItemInt(hDlg, IDC_EDIT1, Value, bSigned) — устанавливается значение переменной целого типа Value в окне редактирования. Если bSigned равно TRUE, рассматривается число со знаком.

Для демонстрации возможностей этого окна создадим тестовую программу (листинг 3.5), вид ее окна приведен на рис. 3.8. В диалоговом окне осуществляется ввод текстовых строк в элемент управления список, а после закрытия диалогового окна данные выводятся в главном окне. Предусмотрена возможность удаления строки списка.

Дополнительно выведем в нижней части главного окна строку состояния (строку статуса), где предусмотрим отображение количества строк списка.

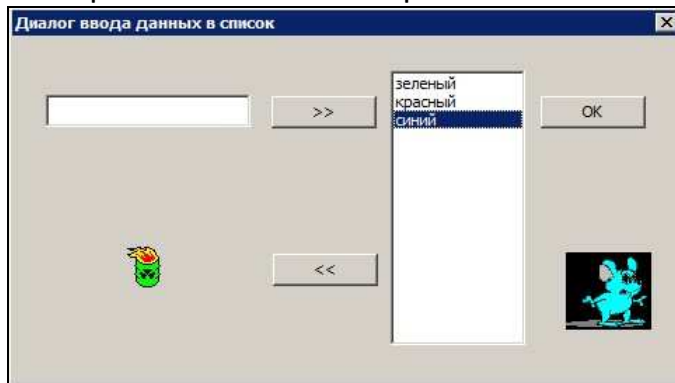


Рис. 3.8. Демонстрационная программа "Окно редактирования"

Листинг 3.5. Окно редактирования и список

```

#include <vector>
#include <xstring>
#include <commctrl.h>
INT_PTR CALLBACK Dialog1(HWND, UINT, WPARAM, LPARAM); typedef
std::basic_string<TCHAR, std::char_traits<TCHAR>, std::allocator<TCHAR> >
String; std::vector<String> v;
HWND hWndStatusBar; //Дескриптор строки состояния
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM
IParam)
{

```

```

    PAINTSTRUCT ps;
    HDC hdc; TEXTMETRIC tm; static int cy, sx, sy; int
y; RECT rt; std::vector<String>::iterator it; switch
(message)
    {
    case WM_CREATE:
        hdc = GetDC(hWnd); GetTextMetrics(hdc,&tm); cy =
tm.tmHeight + tm.tmExternalLeading; ReleaseDC(hWnd, hdc);
        hWndStatusBar = CreateStatusWindow(WS_CHILD | WS_VISIBLE |
        WS_CLIPSIBLINGS | CCS_BOTTOM | SBARS_SIZEGRIP, _T("Ready"), hWnd, 1); break; case
WM_SIZE:
        sx = LOWORD(lParam); sy =
HIWORD(lParam);
        GetWindowRect(hWndStatusBar, &rt); y = rt.bottom-
rt.top;
        MoveWindow(hWndStatusBar, 0, sy - y, sx, sy, TRUE); break; case
WM_COMMAND:
        switch (LOWORD(wParam))
        {
        case ID_DIALOG_READLISTBOX:
            DialogBox(hInst, MAKEINTRESOURCE(IDD_READ), hWnd, Dialog1); break;
        case IDM_EXIT: DestroyWindow(hWnd); break; default: return
DefWindowProc(hWnd, message, wParam, lParam);
        }
        break; case WM_PAINT:
            hdc = BeginPaint(hWnd, &ps);
            for (y = 0, it = v.begin(); it != v.end(); ++it, y += cy)
                TextOut(hdc, 0, y, it->data(), it->size()); EndPaint(hWnd, &ps);
break;
        case WM_DESTROY: PostQuitMessage(0); break; default: return
DefWindowProc(hWnd, message, wParam, lParam);
        }
        return 0;
    }
    //////////////////////////////////////
    INT_PTR CALLBACK Dialog1(HWND hDlg, UINT message, WPARAM wParam, LPARAM lParam)
    {
        static HWND hList, hEdit; TCHAR text[256];
int i, k; switch (message)
    {
        case WM_INITDIALOG:
            hList = GetDlgItem(hDlg, IDC_LIST1);
            hEdit = GetDlgItem(hDlg, IDC_EDIT1); return TRUE; case
WM_COMMAND:
            switch (LOWORD(wParam))
            {
                case IDC_ADD:
                    GetDlgItemText(hDlg, IDC_EDIT1, text, 256);
                    SetDlgItemText(hDlg, IDC_EDIT1, _T(""));
                    SendMessage(hList, LB_ADDSTRING, 0, (LPARAM)text); SetFocus(hEdit);
return TRUE; case IDC_DEL:

```

```

        k = SendMessage(hList, LB_GETCURSEL, 0, 0);  SendMessage(hList,
LB_DELETESTRING, (WPARAM)k, 0);  return TRUE;  case ID_OK:
    v.clear();
    k = SendMessage(hList, LB_GETCOUNT, 0, 0);  for (i = 0; i < k; i++)
    {
        SendMessage(hList, LB_GETTEXT, (WPARAM)i, (LPARAM)text);  v.push_back(text);
    }
    InvalidateRect(GetParent(hDlg), NULL, 1);
    _stprintf(text, _T("Список: %d строк"), k);
    SendMessage(hWndStatusBar, WM_SETTEXT, 0, (LPARAM)text);  case WM_DESTROY:
EndDialog(hDlg, LOWORD(wParam)); return TRUE;
    }
    default: return FALSE;
    }
    return FALSE;
}

```

В качестве контейнера для хранения текстовых строк, получаемых из списка, используем контейнер vector для типа данных String, производного от шаблонного класса basic_string для типа TCHAR (см. листинг 1.3). В заголовке программы добавим файлы включений:

```

#include <vector>
#include <xstring>

```

Опишем на глобальном уровне вектор, явно указывая область видимости: std::vector<String> v;

Итератор вектора нет смысла описывать на глобальном уровне, поэтому опишем его в функции главного окна: std::vector<String>::iterator it;

Нужно позаботиться о выводе результатов в главное окно, поэтому в сообщении WM_CREATE найдем высоту строки текущего шрифта при помощи функции GetTextMetrics(), как мы это делали в листинге 1.13.

Переменную tm типа TEXTMETRIC и статическую переменную су, необходимую для хранения высоты шрифта, предварительно опишем в оконной функции.

Диалоговое окно будем вызывать через главное меню, где создадим пункт с идентификатором ID_DIALOG_READLISTBOX. Dialog1 — имя оконной функции диалога. Именно в этой функции и реализуется вся логика работы с элементами управления.

На поверхности диалогового окна разместим следующие элементы:

- ☐ поле ввода (EditBox) с идентификатором IDC_EDIT1;
- ☐ список (ListBox) с идентификатором IDC_LIST1;
- ☐ кнопку IDC_ADD;
- ☐ кнопку IDC_DEL;
- ☐ кнопку ID_OK;
- ☐ иконку IDC_STATIC с изображением IDI_TRASH;  битовый образ с изображением IDB_MOUSE.

Последние два элемента мы ввели, чтобы "оживить" диалоговое окно, а картинки нашли в имеющейся коллекции и импортировали в созданный проект (см. рис. 3.2). Разместим их в элементе управления Picture Control.

Начнем рассмотрение функции диалогового окна Dialog1().

В сообщении WM_INITDIALOG определим дескрипторы окна редактирования и списка:

```

hList = GetDlgItem(hDlg, IDC_LIST1); hEdit =
GetDlgItem(hDlg, IDC_EDIT1);

```

Эти переменные описаны в заголовке функции: static HWND hList, hEdit;

Теперь обработаем реакцию на нажатие кнопки с изображением ">>".

Функцией `GetDlgItemText()` читаем содержимое окна редактирования в массив `text` и функцией `SetDlgItemText()` очищаем окно, посылая "пустую" строку. Посылая списку сообщение `LB_ADDSTRING`, добавим к нему строку `text`. Теперь установим фокус ввода обратно на окно редактирования для последующего ввода: `SetFocus(hEdit);`

ПРИМЕЧАНИЕ

Сообщение `LB_ADDSTRING` позволяет добавлять строки текста в конец списка, если для него не установлено свойство `Sort`. Если же свойство установлено, то строки будут добавляться в порядке, установленном критерием сортировки по умолчанию.

Обработаем реакцию на нажатие кнопки "<<" для удаления выделенной в списке строки. Посылая списку сообщение `LB_GETCURSEL`, получим индекс выделенного элемента, а сообщением `LB_DELETESTRING` удалим элемент с найденным индексом.

Нажатию на кнопку **ОК** (идентификатор которой заменим на `ID_OK`, и установим в `False` свойство `Default Button`, чтобы избежать стандартной реакции на нажатие клавиши <Enter>) извлечем из списка введенные строки и поместим их в контейнер `vector`, описанный на глобальном уровне. Очистим содержимое контейнера:

```
v.clear();
```

Найдем размер списка, передавая ему сообщение `LB_GETCOUNT`: `k =`
`SendMessage(hList, LB_GETCOUNT, 0, 0);`

В цикле читаем последовательно содержимое списка, посылая ему сообщение `LB_GETTEXT`, где третьим параметром служит индекс извлекаемого элемента, а четвертый параметр — массив `text` для хранения строки текста. Прочитав строку текста, помещаем ее в контейнер методом `push_back()`. `TCHAR`-строка автоматически преобразуется к типу `String`.

```
for (i = 0; i < k; i++)  
{  
    SendMessage(hList, LB_GETTEXT, (WPARAM)i, (LPARAM)text);  
    v.push_back(text); }
```

ПРИМЕЧАНИЕ

В этой задаче применение контейнера для хранения строк не является необходимым, поскольку количество строк в списке известно, и можно было бы выделить массив типа `String`.

Функцией `InvalidRect()` инициируем перерисовку главного окна программы, где дескриптор главного окна получим обращением к функции `GetParent(hDlg)`.

Мы не поставили оператор `return` перед следующим оператором `case` намеренно, поскольку выполнение кода будет продолжаться, произойдет переход на операторы закрытия диалогового окна и выхода из функции:

```
EndDialog(hDlg, LOWORD(wParam)); return TRUE; что и требовалось.
```

Сообщение с кодом `WM_DESTROY` будет генерироваться при нажатии на кнопку завершения приложения системного меню. В этом случае приложение завершится без сохранения данных списка и перерисовки главного окна.

Мы поместили на диалоговом окне еще два элемента **Picture Control**, оставив для них идентификатор по умолчанию `IDC_STATIC`, поскольку нет необходимости доступа к ним. Для первого элемента выбрали тип `Icon`, а в качестве изображения выбрали идентификатор `IDI_TRASH` импортированной иконки, которую подобрали в специализированной библиотеке.

Для второго элемента выберем тип `Bitmap`, а в качестве битового образа — `IDB_MOUSE`. Этот идентификатор мы получим, импортируя `bmp`-файл с растровым изображением. Для полноты картины заменим и иконку приложения. Для этого импортируем еще одну иконку из найденной коллекции и присвоим ей идентификатор `IDI_FLGRUS`.

Теперь осталось изменить стиль окна, для чего отредактируем лишь одно поле класса окна: `wcx.hIconSm = LoadIcon(wcx.hInstance, MAKEINTRESOURCE(IDI_FLGRUS));` Вот и все, теперь в качестве иконки приложения будет использоваться найденное нами изображение.

Строка состояния

Для того чтобы придать приложению, представленному в листинге 3.5, более современный вид, добавим в окно строку состояния. Эта строка располагается обычно в нижней части окна и выводит справочную информацию о состоянии приложения. Выведем здесь количество строк списка.

Поскольку окно состояния является общим элементом управления, необходимо добавить файл включений `commctrl.h` и, соответственно, библиотечный файл `comctl32.lib`.

Дескриптор окна состояния опишем на глобальном уровне:

`HWND hWndStatusBar`; и создадим окно при обработке сообщения `WM_CREATE` главного окна функцией `CreateStatusWindow()`:

`HWND WINAPI CreateStatusWindowW(LONG style, LPCWSTR lpszText, HWND hwndParent, UINT wID)`; где:

- `style` — стиль окна;
- `lpszText` — текст по умолчанию;
- `hwndParent` — дескриптор родительского окна;
- `wID` — идентификатор окна.

К сожалению, окно состояния не может самостоятельно подстроиться под размер окна при его изменении, поэтому необходимо написать дополнительный код при обработке сообщения `WM_SIZE`:

```
GetWindowRect(hWndStatusBar, &rt); y = rt.bottom-rt.top;  
MoveWindow(hWndStatusBar, 0, sy - y, sx, sy, TRUE);
```

Теперь нужно решить вопрос — где мы будем выводить информацию в строку состояния? Проще всего это сделать при обработке сообщения о нажатии кнопки **ОК** диалогового окна. Для этого мы сформируем строку вывода `text`:

`_sprintf(text, _T("Список: %d строк"), k)`; и выведем эту строку в окно состояния, пошлав ему сообщение `WM_SETTEXT`:

```
SendMessage(hWndStatusBar, WM_SETTEXT, 0, (LPARAM)text);
```

Простой текстовый редактор на элементе управления

Edit Box Control

Мы рассмотрели использование окна редактирования `Edit Box` для ввода строки текста, однако этот элемент управления имеет гораздо больше возможностей. В качестве демонстрационного примера построим на его основе простой текстовый редактор. Как мы вскоре убедимся, все операции по вводу и редактированию текста элемент `Edit Box` берет на себя, нам остается лишь реализовать внешний интерфейс. Следует оговориться, что элемент управления `Edit Box` использует внутреннюю память для хранения текста, а это приводит к ограничениям на размер редактируемого текста в 32 767 символов (0x7fff).

ПРИМЕЧАНИЕ

В операционной системе Windows NT и выше размер буфера `Edit Box` может быть увеличен посылкой ему сообщения `EM_LIMITTEXT` до 0x7FFFFFFE байтов.

В качестве основы используем стандартный проект `Win32`. Добавим три пункта меню: **New**, **Open**, **Save**, панель инструментов с кнопками, соответствующими этим пунктам меню, а также строку состояния (см. листинг 3.6). Будем создавать универсальный проект, однако, поскольку приложение ориентировано на работу с текстовыми файлами в однобайтной кодировке, будем явно указывать тип `char*` для входных и выходных массивов данных.

Листинг 3.6. Текстовый редактор с элементом управления Edit Box

```
#include <commdlg.h>  
#include <commctrl.h>  
#include <fstream>  
TBBUTTON tbb[] =  
{ {STD_FILENEW, ID_FILE_NEW, TBSTATE_ENABLED, TBSTYLE_BUTTON, 0, 0, 0, 0},
```

```

{STD_FILEOPEN, ID_FILE_OPEN, TBSTATE_ENABLED, TBSTYLE_BUTTON, 0, 0, 0, 0},
{STD_FILESAVE, ID_FILE_SAVE, TBSTATE_ENABLED, TBSTYLE_BUTTON, 0, 0, 0, 0} };
////////////////////////////////////
VOID StatusOut(HWND hStatus, int count, TCHAR *str)
{
    TCHAR text[256];
    _stprintf(text, _T("Строк: %d"), count);
    SendMessage(hStatus, SB_SETTEXT, (WPARAM)0, (LPARAM)text);
    SendMessage(hStatus, SB_SETTEXT, (WPARAM)1, (LPARAM)str);
}
////////////////////////////////////
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM
IParam)
{
    static OPENFILENAME file; static int n, sx,
sy;
        static HWND hEdit, hWndToolBar, hWndStatusBar; RECT rt; int
m, k, aWidths[2]; static TCHAR name[256];
        char szText[0x7fff]; std::ifstream in;
std::ofstream out; switch (message)
    {
        case WM_CREATE:
            hWndToolBar = CreateToolBarEx(hWnd, WS_CHILD|WS_VISIBLE|CCS_TOP, 2, 0,
HINST_COMMCTRL, IDB_STD_SMALL_COLOR, tbb, 3, 0, 0, 0, 0, sizeof(TBBUTTON));
            hEdit = CreateWindow(WC_EDIT, NULL, WS_CHILD|WS_VISIBLE|WS_HSCROLL|
WS_VSCROLL|ES_LEFT|ES_MULTILINE|ES_AUTOHSCROLL|ES_AUTOVSCROLL,
0, 0, 0, 0, hWnd, (HMENU) 1, hInst, NULL); file.lStructSize =
sizeof(OPENFILENAME); file.hInstance = hInst;
            file.lpstrFilter = _T("Text\0 *.txt\0Все файлы\0 *.*"); file.lpstrFile =
name; file.nMaxFile = 256;
            file.lpstrInitialDir = _T(".\\"); file.lpstrDefExt = _T("txt");
            hWndStatusBar = CreateWindow(STATUSCLASSNAME, NULL, WS_CHILD |
WS_VISIBLE, 0, 0, 0, 0, hWnd, NULL, hInst, NULL); break; case WM_SIZE:
            sx = LOWORD(IParam); sy =
HIWORD(IParam); aWidths[0] = 100; aWidths[1] =
sx;
            GetWindowRect(hWndToolBar, &rt); m = rt.bottom -
rt.top;
            SendMessage(hWndToolBar, TB_AUTOSIZE, 0, 0);
GetWindowRect(hWndStatusBar, &rt); k = rt.bottom - rt.top;
            MoveWindow(hWndStatusBar, 0, sy - k, sx, sy, TRUE);
            SendMessage(hWndStatusBar, SB_SETPARTS, (WPARAM)2, (LPARAM)aWidths);
            StatusOut(hWndStatusBar, n, name);
            MoveWindow(hEdit, 0, m, sx, sy - m - k, TRUE);
            UpdateWindow(hEdit); SetFocus(hEdit);
return 0;
        case WM_COMMAND:
            switch (LOWORD(wParam))
            {
                case ID_FILE_NEW:
                    szText[0] = '\0';
                    SetWindowTextA(hEdit, szText); StatusOut(hWndStatusBar, 0, _T(""));

```



```

        break; case ID_FILE_OPEN:
            file.lpszTitle = _T("Открыть файл для чтения"); file.Flags =
OFN_HIDEREADONLY; if (!GetOpenFileName(&file)) return 1; in.open(name,
std::ios::binary); in.read(szText, 0x7fff); if ((m = in.gcount()) == 0x7fff)
    {
        MessageBox(hWnd, _T("Слишком большой файл"),
_T("Edit"), MB_OK | MB_ICONSTOP); in.close(); return 0;
    }
    szText[m] = '\0'; in.close();
    SetWindowTextA(hEdit, szText);
    n = SendMessage(hEdit, EM_GETLINECOUNT, 0, 0); StatusOut(hWndStatusBar, n, name);
    break; case ID_FILE_SAVE:
        file.lpszTitle = _T("Открыть файл для записи"); file.Flags =
OFN_NOTESTFILECREATE | OFN_HIDEREADONLY; if (!GetSaveFileName(&file)) return
1; out.open(name, std::ios::binary); m = GetWindowTextA(hEdit, szText, 0x7fff);
out.write(szText, m); out.close();
    n = SendMessage(hEdit, EM_GETLINECOUNT, 0, 0); StatusOut(hWndStatusBar, n, name);
    break;
    case IDM_EXIT: DestroyWindow(hWnd); break; default: return
DefWindowProc(hWnd, message, wParam, lParam);
    }
    break;
    case WM_DESTROY: PostQuitMessage(0);
break;
    default: return DefWindowProc(hWnd, message, wParam, lParam);
} return 0;
}

```

Нам понадобятся файлы включений: `commdlg.h`, `commctrl.h`, `fstream`. Доступ к стандартной области имен открывать не будем, а опишем переменные с явным указанием области видимости.

Как и в программе просмотра файлов, рассмотренной в *главе 2* (см. листинг 2.3), опишем в массиве `TBBUTTON` 3 кнопки инструментальной панели. Саму панель инструментов создадим при обработке сообщения `WM_CREATE` и получим ее дескриптор `hwndToolBar`:

```

hwndToolBar = CreateToolBarEx(hWnd, WS_CHILD|WS_VISIBLE|CCS_TOP, 2, 0,
HINST_COMMCTRL, IDB_STD_SMALL_COLOR, tbb, 3, 0, 0, 0, 0, sizeof(TBBUTTON));

```

Корректировку размера панели инструментов вставим в обработчик сообщения `WM_SIZE`, посылая ей сообщение `TB_AUTOSIZE`:

```
SendMessage(hwndToolBar, TB_AUTOSIZE, 0, 0);
```

Здесь же создадим элемент управления Edit Box:

```

hEdit = CreateWindow(WC_EDIT, NULL, WS_CHILD|WS_VISIBLE|WS_HSCROLL|
WS_VSCROLL|ES_LEFT|ES_MULTILINE|ES_AUTOHSCROLL|ES_AUTOVSCROLL,
0, 0, 0, 0, hWnd, (HMENU) 1, hInst, NULL);

```

В параметре *Класс окна* функции `CreateWindow()` указываем `WC_EDIT` — предопределенный в системе идентификатор окна Edit Box. Заголовка у окна нет, поэтому второй параметр `NULL`.

Стиль окна будет складываться из следующих компонент:

- ☐ `WS_CHILD` — дочернее окно;
- ☐ `WS_VISIBLE` — окно отображается при создании;
- ☐ `WS_HSCROLL` — имеет горизонтальную полосу скроллинга;
- ☐ `WS_VSCROLL` — имеет вертикальную полосу скроллинга;
- ☐ `ES_LEFT` — выравнивание текста влево;
- ☐ `ES_MULTILINE` — многострочное окно редактирования;

- ❑ ES_AUTOHSCROLL — автоматическая прокрутка текста при вводе строки;
- ❑ ES_AUTOVSCROLL — автоматическая прокрутка текста, когда окно заполнено.

Все четыре параметра расположения и размера окна зададим равным 0, а реальные размеры определим позднее в сообщении WM_SIZE.

Далее укажем дескриптор родительского окна hWnd. Следующий параметр — указатель меню — в данном контексте служит для задания идентификатора окна: присвоим ему номер 1. Укажем следующим параметром дескриптор приложения — hInst и последний параметр — NULL, поскольку дополнительные параметры отсутствуют.

Окно создано, но мы хотели бы наложить его на главное окно программы, размеры которого нужно определять в сообщении WM_SIZE. Сделаем это функцией:

```
MoveWindow(hEdit, 0, m, sx, sy - m - k, TRUE);
```

Функция MoveWindow() позволяет изменить положение (0, m) и размер окна (sx, sy - m - k). Поскольку окно редактирования объявлено как дочернее, то располагается оно в клиентской области главного окна и начало его нужно поместить в левом верхнем углу клиентской области. Но там располагается панель инструментов, высоту которой мы определим, обратившись к функции GetWindowRect():

```
GetWindowRect(hwndToolBar, &rt); m = rt.bottom - rt.top;
```

Таким образом, левый верхний угол окна редактирования нужно сместить вниз по оси y на m. Высоту окна следует уменьшить на вертикальный размер панели инструментов и строки состояния, которая также располагается в клиентской области главного окна: sy - m - k.

Строку состояния создадим с нулевыми размерами функцией CreateWindow() в сообщении WM_CREATE:

```
hWndStatusBar = CreateWindow(STATUSCLASSNAME, NULL, WS_CHILD |  
WS_VISIBLE, 0, 0, 0, 0, hWnd, NULL, hInst, NULL);
```

ПРИМЕЧАНИЕ

Идентификатор строки состояния STATUSCLASSNAME определен в файле включений как: "msctls_statusbar32".

Реальный же размер зададим в сообщении WM_SIZE:

```
MoveWindow(hWndStatusBar, 0, sy - k, sx, sy, TRUE);
```

 где sx, sy — ширина и высота главного окна, а k — высота окна состояния.

Воспользуемся некоторыми особенностями этого окна управления. Имеется возможность разделить строку состояния на несколько частей и выводить информацию в каждую часть отдельно. Для этого необходимо, послав сообщение SB_SETPARTS, указать в wParam количество частей, а в lParam их правые границы, заданные массивом aWidths:

```
SendMessage(hWndStatusBar, SB_SETPARTS, (WPARAM)2, (LPARAM)aWidths);
```

 Поделим строку состояния на две части, установив фиксированный размер первой части, а для второй части отдадим остаток строки:

```
aWidths[0] = 100; aWidths[1] = sx;
```

Вывод в строку состояния организуем в локальной функции StatusOut(), куда передаем количество строк текста count и имя файла str.

Сформируем вывод в первую часть строки состояния функцией _sprintf() и отправим данные, послав окну сообщение SB_SETTEXT. Номер части указывается в wParam сообщения. Во вторую часть поместим имя файла str, полученное из диалогового окна открытия файла.

После установки всех размеров необходимо обратиться к функции:

```
UpdateWindow(hEdit);
```

 иначе окно не успеет перерисовываться.

Что еще важно, так это обеспечить получение фокуса ввода окном редактирования:

```
SetFocus(hEdit);
```

Собственно, что касается окна редактирования, то оно уже должно работать — можно вводить текст и редактировать его, доступны скроллинг и контекстное меню. Оставшийся код необходим для организации чтения файла в окно редактирования и записи в файл отредактированного текста.

Как и в задаче, представленной листингом 2.3, в сообщении о создании окна заполним структуру `OPENFILENAME`. И лишь два поля этой структуры: `lpstrTitle` и `Flags` отличающихся при чтении и записи, мы заполним при обработке команд чтения и записи файла. Диалог выбора файла реализуется функцией `GetOpenFileName()`.

Но здесь имеется одна особенность — дело в том, что окно редактирования имеет собственную память, и нам нет необходимости вводить контейнер для хранения текста, вместо этого нужно поместить прочитанный файл в окно редактирования. Нужно учесть еще и ограничение — максимальный размер текста в окне редактирования 32K.

ПРИМЕЧАНИЕ

Размер буфера окна редактирования `Edit Box` может быть увеличен посылкой ему сообщения `EM_SETLIMITTEXT`.

И еще один момент — строки в окне редактирования разделяются парой символов `"\r\n"` так же, как и в файле, и это не случайно.

Можно, конечно, организовать чтение файла построчно и добавлять эти строки в окно редактирования, однако проще прочитать файл как двоичный в буфер `szText` и оттуда отправить весь текст в окно, так мы и поступим.

Итак, суммируя все сказанное: открываем файл как двоичный и читаем блоком не более `0x7fff` символов.

```
in.open(name, std::ios::binary); in.read(szText, 0x7fff);
```

Если прочитали ровно `0x7fff` символов, то файл имеет больший размер и не может быть помещен в окно редактирования. Мы выводим предупреждающее сообщение и прекращаем чтение:

```
if ((m = in.gcount()) == 0x7fff)
{
    MessageBox(hWnd, _T("Слишком большой файл"), _T("Edit"), MB_OK | MB_ICONSTOP);
in.close(); return 0;
}
```

Если же размер файла меньше критического, добавим признак завершения строки `'\0'`, поскольку метод `read()` возвращает массив символов, а нам нужна C-строка: `szText[m] = '\0'`;

После чего закрываем файл.

Теперь поместим текст в виде строки в окно редактирования. Это делается функцией:

```
SetWindowText(hEdit, szText);
```

Осталось только подсчитать количество текстовых строк, передавая окну редактора сообщение `EM_GETLINECOUNT`, и обновить строку состояния.

Для сохранения файла, при обработке пункта меню **Save**, нужно решить обратную задачу. Так же открываем двоичный файл для записи, но метод для записи блока требует задания его размера, поэтому при копировании текста из окна редактирования в буфер функцией `GetWindowText()` сохраняем его размер `m` в байтах. После чего записываем буфер в файл.

```
out.open(name, std::ios::binary); m =
GetWindowText(hEdit, szText, 0x7fff); out.write(szText, m);
out.close();
```

Вновь определяем количество строк текста и обновляем строку состояния.

Еще проще выглядит обработка команды меню **New**. Мы сделаем буфер текста "пустым", просто присвоив первому байту массива символ `'\0'`. Теперь осталось послать эту "пустую" строку в окно редактирования, и окно очистится:

```
szText[0] = '\0';
SetWindowText(hEdit, szText);
```

Теперь нужно очистить строку состояния. Вот и все! Вид работающего приложения представлен на рис. 3.9. Вся логика ввода и редактирования текста, а также скроллинг и контекстное меню уже реализованы в окне редактирования, и нам осталось лишь пожинать плоды выполненной работы.

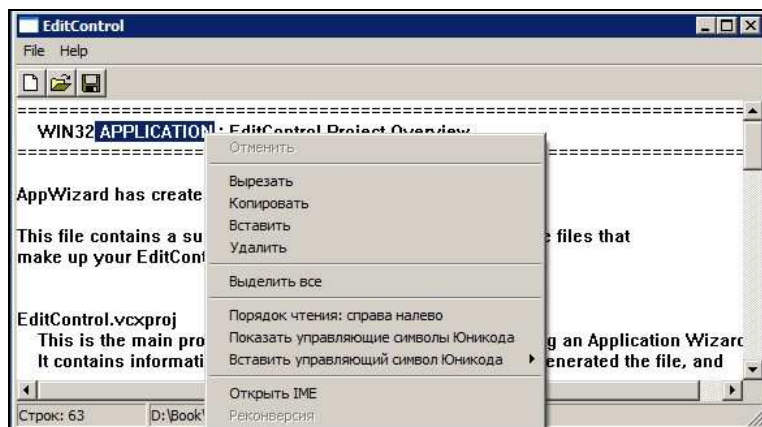


Рис. 3.9. Простейший редактор текстов с элементом управления Edit Box Control

Немодальные окна

До сих пор мы имели дело с модальными окнами, которые после создания становятся активными, и пока они открыты, управление не может быть передано другому окну приложения. Немодальные же окна работают принципиально иначе — после создания они могут как потерять управление, так и получить его вновь. Примером такого окна может служить диалоговое окно поиска **Найти и заменить** (Find and Replace) в текстовом редакторе Word.

Для демонстрации работы немодального окна построим приложение с диалоговым окном выбора цвета фона главного окна, состоящего из трех линейек скроллинга, для базовых цветов: красного, зеленого, синего (рис. 3.10).

В качестве основы используем стандартную заготовку, куда добавим пункт меню **Color**, при обработке которого создадим диалоговое окно, но, поскольку нам нужно немодальное окно, создавать окно будем функцией `CreateDialog(): hDlgColor = CreateDialog(hInst, MAKEINTRESOURCE(IDD_COLOR), hWnd, Dialog);`

Функция имеет тот же набор параметров, что и `DialogBox()`, но возвращает дескриптор окна, который мы опишем на глобальном уровне. На самом деле это макрос, который преобразуется в функцию `CreateDialogParam()`.

ПРИМЕЧАНИЕ

При создании диалогового окна нужно установить для него свойство `Visible`. Иначе окно, созданное функцией `CreateDialog()`, не будет отображаться.

Поскольку немодальное окно может потерять управление, то цикл обработки сообщений головной функции `WinMain()` должен определить источник сообщения, которое может исходить либо от главного, либо немодального окна. Для этого добавим оператор `if` в цикл обработки сообщений:

```
while (GetMessage(&msg, NULL, 0, 0))
{
```

```
if (!TranslateAccelerator(msg.hwnd, hAccelTable, &msg))
```

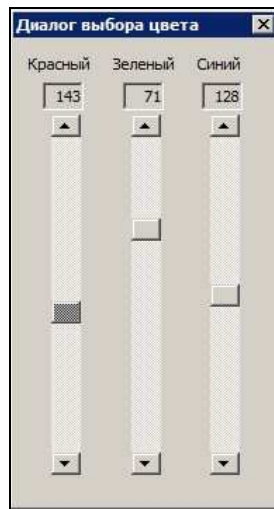


Рис. 3.10. Немодалное диалоговое окно

```
if (!IsDialogMessage(hDlgColor, &msg))
```

```
{
    TranslateMessage(&msg);
    DispatchMessage(&msg);
}
```

Функция `IsDialogMessage()` определяет, предназначено ли сообщение для указанного диалогового окна и, если это так, обрабатывает сообщение, возвращая `TRUE`, иначе обработка сообщения происходит традиционным образом оконной функцией главного окна. Так происходит "диспетчеризация" сообщений и, например, щелчок мыши на диалоговом окне приведет к обработке этого сообщения функцией диалогового окна, а такой же щелчок в главном окне будет обрабатываться функцией главного окна. *В связи с этим принципиально важно, чтобы для всех сообщений, обрабатываемых в оконной функции немодалного диалога, возвращаемое значение было `TRUE`, а для всех прочих сообщений — `FALSE`.*

Далее приведем листинг 3.7 с текстом функции окна немодалного диалогового окна и отметим, что закрываться немодалное окно должно функцией `DestroyWindow()`.

Листинг 3.7. Оконная функция немодалного диалогового окна

```
INT_PTR CALLBACK Dialog(HWND, UINT, WPARAM, LPARAM);
////////////////////////////////////
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM
lParam) { switch (message)
{
    case WM_COMMAND:
        switch (LOWORD(wParam))
        {
            case ID_DIALOG_COLOR: hDlgColor = CreateDialog(hInst,
MAKEINTRESOURCE(IDD_COLOR), hWnd, Dialog); break;
            case IDM_EXIT: DestroyWindow(hWnd); break; default: return
DefWindowProc(hWnd, message, wParam, lParam);
        }
        break;
    case WM_DESTROY: PostQuitMessage(0); break; default: return
DefWindowProc(hWnd, message, wParam, lParam);
}
```

```

    return 0;
}
////////////////////////////////////
INT_PTR CALLBACK Dialog(HWND hDlg, UINT message, WPARAM wParam, LPARAM lParam)
{
    static int ID_SCROLL[3] = {IDC_RED, IDC_GREEN, IDC_BLUE}; static int ID[3] =
{IDC_R, IDC_G, IDC_B}; static HWND hWnd, hScroll[3];    //{hRed, hGreen, hBlue} static
int color[3] = {255, 255, 255}; //{red, green, blue} HBRUSH hBrush; int index; switch
(message)
    {
    case WM_INITDIALOG:
        for (index = 0; index < 3; index++)
        {
            hScroll[index] = GetDlgItem(hDlg, ID_SCROLL[index]);
            SetScrollRange(hScroll[index], SB_CTL, 0, 255, FALSE);
            SetScrollPos (hScroll[index], SB_CTL, color[index], TRUE);
            SetDlgItemInt(hDlg, ID[index], color[index], 0);
        }
        return TRUE; case WM_VSCROLL :
        for (index = 0; index < 3; index++) if ((HWND)lParam ==
hScroll[index]) break; SetFocus(hScroll[index]);
        switch(LOWORD(wParam))
        {
            case SB_LINEUP : color[index]--; break; case SB_LINEDOWN :
color[index]++; break; case SB_PAGEUP : color[index] -= 16; break; case
SB_PAGEDOWN : color[index] += 16; break; case SB_THUMBTRACK:
            case SB_THUMBPOSITION : color[index] = HIWORD(wParam); break;
        }
        color[index] = max(0, min(color[index], 255)); if (color[index] !=
GetScrollPos(hScroll[index], SB_CTL))
        {
            SetDlgItemInt(hDlg, ID[index], color[index], 0);
            SetScrollPos(hScroll[index], SB_CTL, color[index], TRUE); hBrush =
CreateSolidBrush(RGB(color[0], color[1], color[2])); hWnd = GetParent(hDlg);
DeleteObject((HBRUSH)GetClassLong(hWnd, GCL_HBRBACKGROUND));
SetClassLong(hWnd, GCL_HBRBACKGROUND, (LONG)hBrush);
            InvalidateRect(hWnd, NULL, TRUE);
        }
        return TRUE; case WM_COMMAND: if (LOWORD(wParam) == IDCANCEL)
{DestroyWindow(hDlg); return TRUE;} return FALSE;
    }
    return FALSE;
}

```

Рассмотрим оконную функцию диалогового окна. Поскольку диалоговое окно содержит 3 линейки скроллинга и 3 статических элемента для отображения состояния скроллинга, то нам удобнее описать эти элементы массивами, присвоив их элементам значение соответствующих идентификаторов:

```

static int ID_SCROLL[3] = {IDC_RED, IDC_GREEN, IDC_BLUE}; static int ID[3] = {IDC_R, IDC_G,
IDC_B};

```


Так же опишем массив дескрипторов для линейек скроллинга и массив для хранения трех базовых цветов:

```
static HWND hScroll[3]; // {hRed, hGreen, hBlue}
```

```
static int color[3] = {255, 255, 255}; // {red, green, blue}
```

Опишем дескриптор окна hWnd и кисти hBrush, а также переменную целого типа index.

Далее организуем обработку сообщений.

При инициализации диалогового окна в цикле определяем дескрипторы линейек скроллинга:

```
hScroll[index] = GetDlgItem(hDlg, ID_SCROLL[index]); Устанавливаем диапазон скроллинга [0;255]:
```

```
SetScrollRange(hScroll[index], SB_CTL, 0, 255, FALSE);
```

и начальную позицию в 255, поскольку начальный цвет окна — белый, соответствует значению RGB(255, 255, 255).

```
SetScrollPos(hScroll[index], SB_CTL, color[index], TRUE); После чего выведем
```

значение в статическом элементе:

```
SetDlgItemInt(hDlg, ID[index], color[index], 0);
```

Теперь рассмотрим сообщение WM_VSCROLL, генерируемое при воздействии на линейку скроллинга. Вначале нужно определить, от какой линейки пришло сообщение. Сделаем это в цикле, сравнивая дескриптор элемента, породившего сообщение, и передаваемого в lParam, с дескрипторами линейек скроллинга:

```
for (index = 0; index < 3; index++) if ((HWND)lParam == hScroll[index]) break;
```

При выходе из цикла переменная index будет соответствовать индексу линейки скроллинга, которая сгенерировала сообщение. Установим фокус ввода на выбранную линейку:

```
SetFocus(hScroll[index]);
```

Это не обязательно, но позволит нам использовать для управления клавиатуру. Обработку сообщений скроллинга осуществим стандартным образом так же, как в листинге 2.3, только вместо переменной будем оперировать элементом массива color[index]:

Убедившись, что состояние скроллинга изменилось, установим новое значение в статическом элементе:

```
SetDlgItemInt(hDlg, ID[index], color[index], 0); а также новую позицию движка линейки:
```

```
SetScrollPos(hScroll[index], SB_CTL, color[index], TRUE);
```

Теперь создаем новую кисть: hBrush = CreateSolidBrush(RGB(color[0],color[1],color[2])); и уничтожаем старую. Дескриптор старой кисти получим обращением к функции GetClassLong():

```
DWORD WINAPI GetClassLong(HWND hWnd, int nIndex); если
```

передать в качестве параметра GCL_HBRBACKGROUND:

```
DeleteObject((HWND)GetClassLong(hWnd, GCL_HBRBACKGROUND));
```

Уничтожить старую кисть необходимо, чтобы не "замусорить" память.

ПРИМЕЧАНИЕ

Здесь нам понадобился дескриптор главного окна hWnd. Можно перенести его определение на глобальный уровень или воспользоваться функцией GetParent().

Осталось установить новое значение параметра класса окна функцией

SetClassLong(), эта функция, напротив, позволит изменить свойства класса окна, зарегистрированного в системе:

DWORD WINAPI SetClassLong(HWND hWnd, int nIndex, LONG dwNewLong); передавая ей тот же параметр GCL_HBRBACKGROUND и дескриптор новой кисти.

```
SetClassLong(hWnd, GCL_HBRBACKGROUND, (LONG)hBrush);
```

После чего объявляем окно недействительным для перерисовки фона:

```
InvalidateRect(hWnd, NULL, TRUE);
```

ПРИМЕЧАНИЕ

Функция GetClassLong() позволяет получить, а SetClassLong() установить большинство параметров класса окна, которые определяются значением передаваемого индекса: GCL_MENUENAME, GCL_HBRBACKGROUND, GCL_HCURSOR, GCL_HICON, GCL_HMODULE, GCL_CBWND, GCL_CBCLSEXTRA, GCL_WNDPROC, GCL_STYLE.

Нам осталось обработать сообщение о закрытии окна. Мы убрали кнопку закрытия диалогового окна и воспользовались кнопкой системного меню, которая приводит к тому же результату.

Все остальные сообщения оконная функция диалога игнорирует, возвращая FALSE.

Вот собственно и все: запустив программу на выполнение, мы можем регулировать цвет фона, передвигая движки линейек скроллинга для трех базовых цветов.

Стандартное диалоговое окно выбора цвета

Картина будет неполной, если мы не рассмотрим стандартное диалоговое окно выбора цвета, реализованное в Windows. Прототипы функции ChooseColor() и структуры CHOOSECOLOR, необходимых для создания диалогового окна, как и для всех других диалоговых окон, описаны в файле включений commdlg.h, который нужно добавить в заголовке программы.

Для создания диалогового окна создадим пункт меню **StdColor** с идентификатором

ID_STDCOLOR. При обработке этого пункта меню вызовем диалоговое окно выбора цвета, которое изображено на рис. 3.11, а код оконной функции, вызывающий это окно, в листинге 3.8.

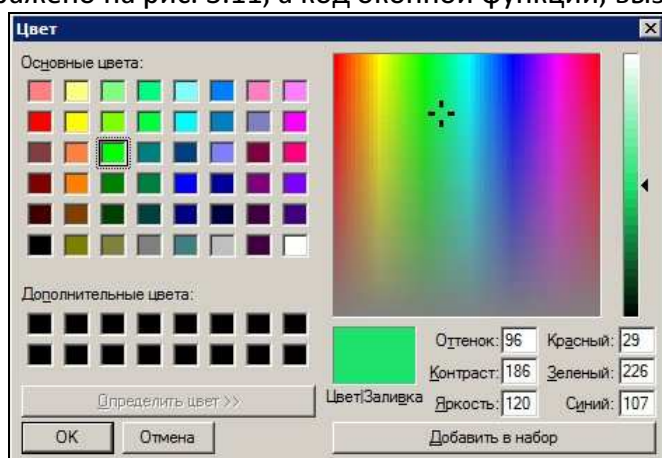


Рис. 3.11. Стандартное диалоговое окно выбора цвета

Листинг 3.8. Оконная функция задачи выбора цвета фона с помощью стандартного диалогового окна

```
#include <commdlg.h>
COLORREF stdColor = RGB(255,255,255);
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM
IParam) { static CHOOSECOLOR ccs; static COLORREF acrCustClr[16]; static HBRUSH hBrush; switch
(message)
{
case WM_CREATE:
ccs.lStructSize = sizeof(CHOOSECOLOR); ccs.hwndOwner =
hWnd; ccs.rgbResult = stdColor; ccs.Flags = CC_RGBINIT |
CC_FULLOPEN; ccs.lpCustColors = (LPDWORD)acrCustClr; break;
case WM_COMMAND:
switch (LOWORD(wParam))
{
case ID_STDCOLOR:
if (ChooseColor(&ccs))
{
stdColor = ccs.rgbResult; if (hBrush) DeleteObject(hBrush);
hBrush = CreateSolidBrush(stdColor);
SetClassLong(hWnd, GCL_HBRBACKGROUND, (LONG)hBrush);
InvalidateRect(hWnd, NULL, TRUE);
}
}
```

```

        break; case IDM_EXIT: DestroyWindow(hWnd); break; default: return
DefWindowProc(hWnd, message, wParam, lParam);
    }
    break;
    case WM_DESTROY: PostQuitMessage(0); break; default: return
DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}

```

Для создания диалогового окна выбора цвета необходимо описать переменную типа структуры CHOOSECOLOR, массив acrCustClr типа COLORREF для хранения 16 дополнительных цветов, а также переменную color, которую окно будет использовать для задания начального цвета.

ПРИМЕЧАНИЕ

В диалоговом окне выбора цвета имеется возможность кроме основного цвета задать еще 16 дополнительных цветов. Причем даже если они не нужны, массив acrCustClr должен быть определен.

Мы задали начальное значение stdColor = RGB(255, 255, 255), чтобы начать выбор с белого цвета.

```

static CHOOSECOLOR ccs;
static COLORREF acrCustClr[16];
Здесь же опишем дескриптор кисти.
static HBRUSH hBrush;

```

Все переменные класса памяти static, чтобы они не потеряли значения при выходе из оконной функции.

Необходимые поля структуры CHOOSECOLOR заполняются при обработке сообщения

```

WM_CREATE. ccs.lStructSize = sizeof(CHOOSECOLOR); //Задается размер структуры
ccs.hwndOwner = hWnd; // Дескриптор родительского окна
ccs.rgbResult = stdColor; //Начальный цвет диалога
ccs.Flags = CC_RGBINIT | CC_FULLOPEN;

```

Флаг работы CC_RGBINIT заставляет диалоговое окно использовать цвет, указанный в rgbResult, как начальный цветовой выбор; флаг CC_FULLOPEN — отображает дополнительные средства управления для создания дополнительных цветов в массиве acrCustClr.

```

ccs.lpCustColors = (LPDWORD)acrCustClr;

```

Диалоговое окно выбора цвета вызывается функцией ChooseColor(), аргументом которой является адрес структуры CHOOSECOLOR. При удачном результате работы функции она возвращает TRUE, а выбранный цвет мы получаем из поля ccs.rgbResult.

Теперь, чтобы не оставлять "мусор" в памяти, уничтожим старую кисть: if (hBrush) DeleteObject(hBrush);

Оператор if используем, чтобы обойти функцию DeleteObject() при первом обращении к диалогу, когда дескриптор hBrush равен NULL. Создаем новую кисть: hBrush = CreateSolidBrush(stdColor); и используем ее в качестве фонового цвета окна:

```

SetClassLong(hWnd, GCL_HBRBACKGROUND, (LONG)hBrush);

```

Осталось только объявить окно недействительным для смены фона:

```

InvalidateRect(hWnd, NULL, TRUE); и задача будет решена.

```

Работа с файлами

Для работы с файлами в API-библиотеке имеется собственный набор функций, однако будем пока работать с классами потокового ввода-вывода, которые хорошо зарекомендовали себя в консольных приложениях. Мы также имеем возможность использовать предоставляемые операционной системой дополнительные сервисы, например, для получения имени файла будем использовать стандартный диалог.

Диалог выбора файлов

Имеется два практически идентичных диалога выбора имени файла: открыть файл для чтения `GetOpenFileName()` и для записи `GetSaveFileName()`:

`BOOL APIENTRY GetOpenFileNameW(LPOPENFILENAMEW);`

`BOOL APIENTRY GetSaveFileNameW(LPOPENFILENAMEW);`

Аргумент функций — указатель на переменную типа `OPENFILENAMEW`. Поля этой структуры приведены в табл. 2.1.

Все функции, обеспечивающие диалог, описаны в файле включений `commdlg.h`.

Таблица 2.1. Описание полей структуры `OPENFILENAMEW`

Тип поля	Имя	Описание
DWORD	<code>lStructSize</code>	Размер структуры в байтах
HWND	<code>hwndOwner</code>	Дескриптор окна-владельца
HINSTANCE	<code>hInstance</code>	Дескриптор приложения
LPCWSTR	<code>lpstrFilter</code>	Фильтр из пар строк — в кодировке Unicode
LPWSTR	<code>lpstrCustomFilter</code>	Пользовательский фильтр — в кодировке Unicode
DWORD	<code>nMaxCustFilter</code>	Размер фильтра в символах
DWORD	<code>nFilterIndex</code>	Смещение в буфере фильтра
LPWSTR	<code>lpstrFile</code>	Указатель на строку с именем файла — в кодировке Unicode
DWORD	<code>nMaxFile</code>	Размер буфера для имени файла
LPWSTR	<code>lpstrFileName</code>	Указатель на строку с выбранным именем файла — в кодировке Unicode

Таблица 2.1 (окончание)

Тип поля	Имя	Описание
DWORD	<code>nMaxFileName</code>	Размер буфера для имени выбранного файла
LPCWSTR	<code>lpstrInitialDir</code>	Указатель на строку начального каталога — в кодировке Unicode
LPCWSTR	<code>lpstrTitle</code>	Указатель на строку с заголовком окна — в кодировке Unicode
DWORD	<code>Flags</code>	Флаг: <code>OFN_ALLOWMULTISELECT</code> — допускается выбор нескольких файлов; <code>OFN_HIDEREADONLY</code> — переключатель <code>ReadOnly</code> не выводится; <code>OFN_READONLY</code> — только для чтения; <code>OFN_OVERWRITEPROMPT</code> — генерируется сообщение при записи существующего файла

WORD	nFileOffset	Смещение имени файла в строке полного имени
WORD	nFileExtension	Смещение расширения имени файла
LPCWSTR	lpstrDefExt	Указатель на строку с расширением имени файла по умолчанию — в кодировке Unicode
LPARAM	lCustData	Данные для функции-перехватчика
LPOFNHOOKPROC	lpfnHook	Указатель на функцию-перехватчик
LPCWSTR	lpTemplateName	Шаблон диалогового окна — в кодировке Unicode

Таким образом, для организации диалога выбора файла необходимо заполнить поля структуры OPENFILENAME и вызвать одну из функций GetOpenFileName() или GetSaveFileName(), после их успешной работы в поле lpstrFile получим указатель на строку с полным именем выбранного файла.

Простой просмотрщик файлов

Рассмотрим простую задачу чтения текстового файла, где для выбора его имени используем стандартный Windows-диалог. В листинге 2.1 приведен текст оконной функции этого приложения.

Листинг 2.1. Просмотрщик файлов с диалогом выбора имени

```
#include <commdlg.h>
#include <fstream>
#include <vector>
#include <string> const int LineHeight = 16; //Высота строки текста + межстрочное расстояние
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    int wmlId;
    PAINTSTRUCT ps;
    HDC hdc;
    static TCHAR name[256] = _T("");;
    static OPENFILENAME file;
    std::ifstream in;
    std::ofstream out;
    static std::vector<std::string> v;
    std::vector<std::string>::iterator it;
    std::string st; int y; switch (message)
    {
        case WM_CREATE:
            file.lStructSize = sizeof(OPENFILENAME);
            file.hInstance = hInst;
            file.lpstrFilter = _T("Text\0*.txt");
            file.lpstrFile = name;
            file.nMaxFile = 256;
            file.lpstrInitialDir = _T(".\\");
            file.lpstrDefExt = _T("txt");
            break;
        case WM_COMMAND: wmlId = LOWORD(wParam); switch (wmlId)
        {
            case ID_FILE_NEW :
                if (!v.empty()) std::vector<std::string>().swap(v);
                InvalidateRect(hWnd, NULL, TRUE);
                break;
        }
    }
}
```

```

case ID_FILE_OPEN :
    file.lpszTitle = _T("Открыть файл для чтения");
file.Flags = OFN_HIDEREADONLY;
if (!GetOpenFileName(&file)) return 1;
in.open(name);
while (getline(in,st)) v.push_back(st);
in.close();
    InvalidateRect(hWnd,NULL,1);
    break;
case ID_FILE_SAVE :    file.lpszTitle = _T("Открыть файл для записи");
file.Flags = OFN_NOTESTFILECREATE;
if (!GetSaveFileName(&file)) return 1;
out.open(name);
    for (it = v.begin(); it != v.end(); ++it) out << *it << '\n';
out.close();
    break;
case IDM_EXIT:
DestroyWindow(hWnd);
    break;
default: return DefWindowProc(hWnd, message, wParam, lParam);
}
    break;
case WM_PAINT:
    hdc = BeginPaint(hWnd, &ps);
    for (y = 0, it = v.begin(); it < v.end(); ++it, y += LineHeight)
        TextOutA(hdc, 0, y, it->data(), it->length());
    EndPaint(hWnd, &ps);
    break;
case WM_DESTROY: PostQuitMessage(0);
    break;
default: return DefWindowProc(hWnd, message, wParam, lParam);
}
return 0;
}

```

Начнем с того, что добавим к существующему меню три пункта. Для этого перейдем на подложку **Resource View** рабочей области и в выпадающем меню **File** введем три пункта **&New**, **&Open**, **&Save**. Если не вводить идентификаторы пунктов меню, то мастер присвоит им значения по умолчанию: ID_FILE_NEW, ID_FILE_OPEN, ID_FILE_SAVE. Затем создадим сепаратор (линию-разделитель): для этого достаточно ввести вместо имени дефис (-) или отметить флаг **Separator** в окне свойств. После чего "отбуксируем" пункт меню **Exit** в конец списка.

ПРИМЕЧАНИЕ

Чтобы вновь не открывать окно свойств для каждого пункта меню, достаточно прикрепить его к "холсту" верхней кнопкой с изображением канцелярской кнопки.

При выборе пункта меню будет сгенерировано сообщение WM_COMMAND, а в младшее слово wParam помещено значение соответствующего идентификатора. Наша задача заключается в добавлении к оконной функции обработчиков этих сообщений. Однако предварительно необходимо решить, как мы будем хранить строки прочитанного файла.

Можно в стиле классического C читать из файла построчно и выделять память для хранения каждой строки, а указатели на строки хранить в одномерном массиве. Но поскольку мы не знаем заранее размер файла, то нам придется либо запастись большой массив указателей, на всякий

случай (что, тем не менее, приведет к ограничению размера файла), либо предварительно определить его размер, а затем выделить необходимую память. Первый вариант для нас совершенно неприемлем, поскольку он абсолютно непрофессионален, второй же вариант потребует читать файл 2 раза, что вряд ли улучшит эффективность программы.

Однако вспомним, что в составе стандартной библиотеки шаблонов *STL* (Standard Template Library) имеется набор контейнеров, которые достаточно просто разрешат наши проблемы. Выберем самый простой контейнер `vector`, где строки будем хранить в виде экземпляров стандартного класса `string`, что существенно упростит работу.

Текстовый файл будем читать, используя потоковые классы ввода/вывода.

Итак, добавляем в заголовок приложения операторы включения соответствующих библиотек `commdlg.h`, `vector`, `string`, `fstream`.

Здесь первая строка необходима для подключения функций файлового диалога, а три следующих файла включений содержат определения классов: `vector`, `string`, `ifstream`, `ofstream`.

ПРИМЕЧАНИЕ

Все потоковые классы и библиотека *STL* определены в стандартной области имен `std`, однако подключение всего пространства имен директивой `"using namespace std"` не является хорошим решением. Для ускорения компиляции рекомендуется явно указывать область видимости для каждой переменной, например, `std::vector<std::string> v`;

Переменные, значение которых должно сохраниться после выхода из оконной функции, мы объявим как `static`:

- ❑ `name` — символьный массив для хранения имени открываемого файла;
- ❑ `file` — переменная типа структуры `OPENFILENAME`;
- ❑ `v` — вектор для данных типа `string`.

Остальные переменные могут быть автоматическими, потому что нет необходимости сохранять их значение между вызовами оконной функции.

Большую часть полей структуры `file` (см. табл. 2.1) разумно заполнить при обработке сообщения `WM_CREATE`, поскольку это сообщение обрабатывается только один раз при создании окна.

ПРИМЕЧАНИЕ

Поскольку переменная `file` описана как `static`, то нулевые значения можно не присваивать — эти поля и так будут инициализированы нулями при создании переменной.

При выборе команды меню **New** мы должны очистить вектор и окно вывода.

Наиболее простой способ для этого — метод `clear()`, однако, очистив вектор, метод не освободит выделенной под него памяти, что не совсем хорошо. Даже если мы воспользуемся методом `v.resize(0)`, то и это не станет решением проблемы — он не сможет уменьшить размер выделенной памяти, поскольку предназначен лишь для увеличения размера контейнера.

Существует довольно элегантный прием для решения этой проблемы: `std::vector<std::string>().swap(v)`;

Здесь создается временный "пустой" вектор для переменных типа `string` и тут же обменивается с вектором `v`. Если вспомнить, что метод `swap()` обменивает не только содержимое контейнеров, но и их размеры, то становится ясно, что в результате мы получим пустой вектор `v` и временный вектор, содержащий предыдущий набор данных, который автоматически уничтожается деструктором.

Разумеется, если вектор и так "пустой", то очищать его нет смысла, поэтому мы используем условный оператор: `if (!v.empty()) . . .`

Теперь, после того как вектор очищен, нужно перерисовать окно, как обычно обратившись к функции `InvalidateRect()`.

По команде **Open** мы заполним лишь два поля переменной `file`:

```
file.lpstrTitle = _T("Открыть файл для чтения");  
file.Flags = OFN_HIDEREADONLY;
```

где первое поле обеспечит вывод заголовка диалогового окна, а второе поле можно было бы и не заполнять, поскольку OFN_HIDEREADONLY означает лишь то, что переключатель **Read Only** не выводится.

При обращении к функции `GetOpenFileName()` происходит вывод диалогового окна (рис. 2.1), где мы имеем возможность выбрать файл для чтения. Если работа функции завершилась успешно, то в символьном массиве `pname` мы получим полное имя файла и сможем открыть его методом `open()` потокового класса.

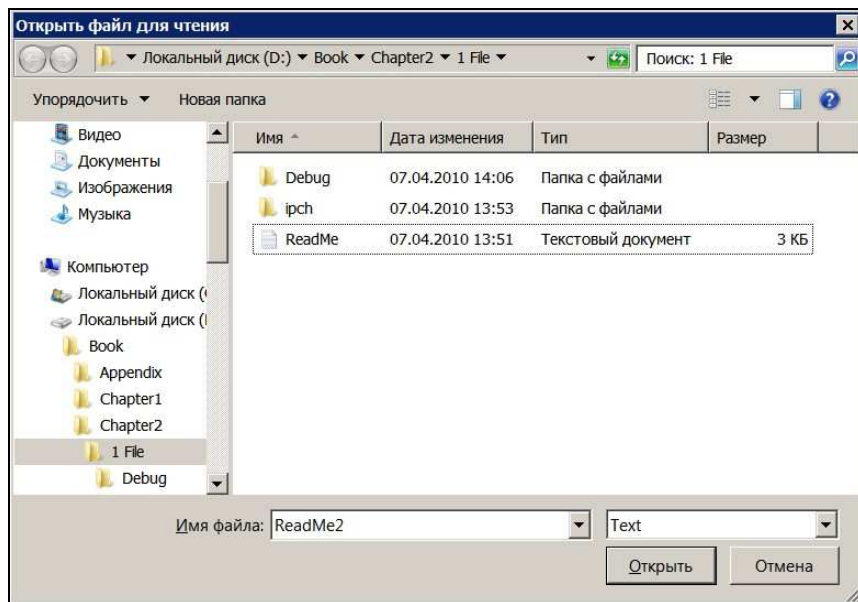


Рис. 2.1. Диалоговое окно выбора имени файла для чтения

Теперь файл построчно читаем в память. Лучше всего воспользоваться перегруженной для класса `string` функцией `getline()`, которая принимает в качестве параметров входной поток и переменную типа `string`. Функция читает из потока `in` строку и помещает ее в переменную `st`, причем конструктор класса `string` сам позаботится о выделении памяти. Далее, методом `v.push_back(st)` строка добавляется в конец контейнера. Возвращаемым значением функции `getline()` является входной поток `in`, поэтому цикл работает, пока весь файл не будет прочитан. При достижении конца файла функция `getline()` возвращает `NULL` и цикл прекращается.

Теперь входной поток нужно закрыть методом `close()` и перерисовать окно.

Обработка команды **Save** осуществляется аналогично. Мы также переопределяем значение двух полей `IpstrTitle` и `Flags`, изменяя заголовок диалогового окна и флаг открытия окна `OFN_NOTESTFILECREATE`, который позволяет выбрать имя существующего файла для замены.

При записи файла мы воспользовались диалогом `GetSaveFileName()`, который отличается от предыдущего только тем, что кнопка в диалоговом окне называется **Сохранить** вместо **Открыть** (рис. 2.2).

Для записи в файл содержимого контейнера организуем цикл от начала до конца контейнера по итератору `it`. Добавим в конце каждой строки символ перевода строки `'\n'`.

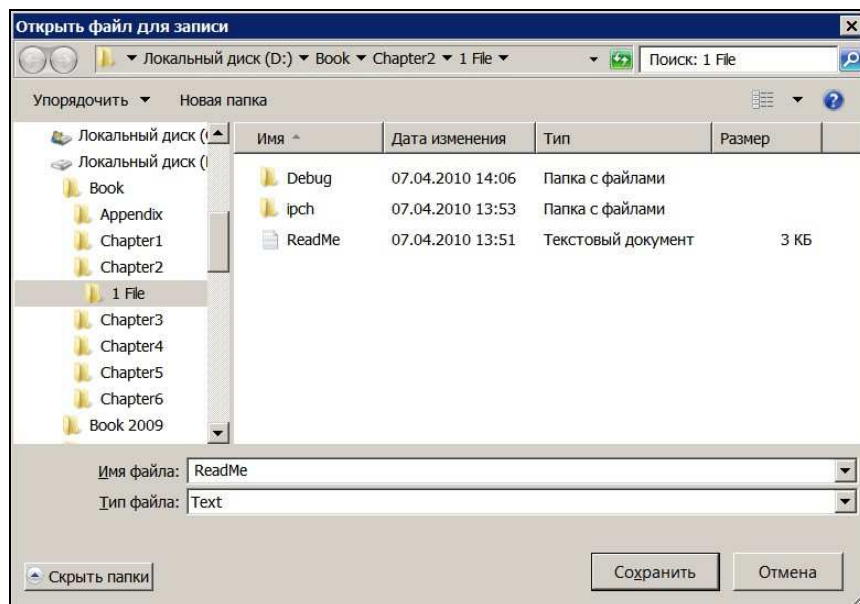


Рис. 2.2. Диалог выбора имени файла для записи

Теперь осталось организовать вывод содержимого контейнера в окно при обработке сообщения WM_PAINT. Мы можем выводить содержимое контейнера в окно по итератору `it`. Нужно только добавить переменную целого типа `y` для позиционирования строк текста по вертикали. Будем увеличивать эту переменную на каждом шаге цикла на `LineHeight` — этого достаточно для шрифта по умолчанию.

Функция `TextOutA()` не требует задания C-строки, ей достаточно символьного массива типа `char*`, однако класс `string` имеет метод `data()`, возвращающий указатель на массив символов. Размер строки текста получим методом `length()`.

ПРИМЕЧАНИЕ

Поскольку приложение ориентировано на чтение файлов с однобайтной кодировкой, то мы должны явно указать суффикс "A" функции `TextOutA()`, что будет означать использование типа `char*` в качестве указателя строки.

Если все сделано правильно, то мы получим программу, читающую текстовый файл, выводящую его в окно и имеющую возможность сохранить его копию.

Приложение хотя и работает, но имеет ряд существенных недостатков:

1. Если весь текст не помещается в окне, часть его будет утеряна.
2. "Исчезают" символы, не имеющие графического образа, в частности символы табуляции.

Решение указанных проблем требует модернизации программы.

Для обработки символов табуляции проще всего заменить функцию вывода текста на `TabbedTextOutA()`:

```
TabbedTextOutA(hdc, 0, y, it->data(), it->length(), 0, NULL, 0);
```

Начальные параметры имеют тот же смысл, а три последних параметра определяют позиции табуляции. По умолчанию (нулевые значения параметров), символ табуляции разворачивается до средней ширины 8 символов.

Организация скроллинга

Скроллинг или прокрутку можно установить как свойство окна, добавив флаги горизонтального и вертикального скроллинга `WS_VSCROLL` и `WS_HSCROLL` в стиль создаваемого окна функции `CreateWindow()`.

```
hWnd = CreateWindow(szWindowClass, szTitle,
    WS_OVERLAPPEDWINDOW | WS_VSCROLL | WS_HSCROLL,
    CW_USEDEFAULT, 0, CW_USEDEFAULT, 0, NULL, NULL, hInstance, NULL);
```

Однако всю логику обслуживания полос прокрутки мы должны организовать самостоятельно (листинг 2.2). Используем в качестве основы предыдущую задачу, листинг 2.1, добавив сюда скроллинг.

Листинг 2.2. Оконная функция с полосами прокрутки

```
#include <commdlg.h>
#include <fstream>
#include <vector>
#include <string>
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM
IParam)
{
    PAINTSTRUCT ps;
    HDC hdc;
    static TCHAR name[256] = _T("");;
    static OPENFILENAME file;
    std::ifstream in;
    std::ofstream out;
    static std::vector<std::string> v;
    std::vector<std::string>::iterator it;
    std::string st; int y, k;
    static int n,length,sx,sy,cx,iVscrollPos,iHscrollPos,COUNT,MAX_WIDTH; static SIZE size = {8, 16};
//Ширина и высота символа switch (message)
{
    case WM_CREATE:
        file.lStructSize = sizeof(OPENFILENAME);
        file.hInstance = hInst;
        file.lpstrFilter = _T("Text\0*.txt");
        file.lpstrFile = name;
        file.nMaxFile = 256;
        file.lpstrInitialDir = _T(".\\");
        file.lpstrDefExt = _T("txt");
        break;
    case WM_SIZE:
        sx = LOWORD(IParam);
        sy = HIWORD(IParam);
        k = n - sy/size.cy;
        if (k > 0) COUNT = k; else COUNT = iVscrollPos = 0;
        SetScrollRange(hWnd, SB_VERT, 0, COUNT, FALSE);
        SetScrollPos(hWnd, SB_VERT, iVscrollPos, TRUE);
        k = length - sx/size.cx;
        if (k > 0) MAX_WIDTH = k; else MAX_WIDTH = iHscrollPos = 0;
        SetScrollRange(hWnd, SB_HORZ, 0, MAX_WIDTH, FALSE);
        SetScrollPos(hWnd, SB_HORZ, iHscrollPos, TRUE); break;
    case WM_VSCROLL :
        switch(LOWORD(wParam))
        {
            case SB_LINEUP : iVscrollPos--; break; case SB_LINEDOWN : iVscrollPos++; break; case
SB_PAGEUP : iVscrollPos -= sy / size.cy; break; case SB_PAGEDOWN : iVscrollPos += sy / size.cy; break;
case SB_THUMBPOSITION : iVscrollPos = HIWORD(wParam); break;
        }
        iVscrollPos = max(0, min(iVscrollPos, COUNT));
        if (iVscrollPos != GetScrollPos(hWnd, SB_VERT))
        {
            SetScrollPos(hWnd, SB_VERT, iVscrollPos, TRUE);
```

```

        InvalidateRect(hWnd, NULL, TRUE);
    }
    break;
case WM_HSCROLL :
    switch(LOWORD(wParam))
    {
        case SB_LINEUP : iHscrollPos--; break;   case SB_LINEDOWN : iHscrollPos++; break;   case
SB_PAGEUP : iHscrollPos -= 8; break;   case SB_PAGEDOWN : iHscrollPos += 8; break;   case
SB_THUMBPOSITION : iHscrollPos = HIWORD(wParam); break;
    }
    iHscrollPos = max(0, min(iHscrollPos, MAX_WIDTH));
    if (iHscrollPos != GetScrollPos(hWnd, SB_HORZ))
    {
        SetScrollPos(hWnd, SB_HORZ, iHscrollPos, TRUE);
        InvalidateRect(hWnd, NULL, TRUE);
    }
    break; case WM_COMMAND:
    switch (LOWORD(wParam))
    {
        case ID_FILE_NEW :   if (!v.empty()) std::vector<std::string>().swap(v);
n = length = 0;
        SendMessage(hWnd, WM_SIZE, 0, sy << 16 | sx);   InvalidateRect(hWnd, NULL, TRUE);
        break;
case ID_FILE_OPEN :
        file.lpszTitle = _T("Открыть файл для чтения");
        file.Flags = OFN_HIDEREADONLY;   if (!GetOpenFileName(&file)) return 1;
in.open(name);
        while (getline(in,st))
        {
            if (length < st.length()) length = st.length();   v.push_back(st);
        }
        in.close();
        n = v.size();
        SendMessage(hWnd, WM_SIZE, 0, sy << 16 | sx);
        InvalidateRect(hWnd, NULL, TRUE);
        break;
case ID_FILE_SAVE :
        file.lpszTitle = _T("Открыть файл для записи");
        file.Flags = OFN_NOTESTFILECREATE;
if (!GetSaveFileName(&file)) return 1;
        out.open(name);
        for (it = v.begin(); it != v.end(); ++it) out << *it << '\n';
out.close();
        break;
        case IDM_EXIT: DestroyWindow(hWnd); break;
default: return DefWindowProc(hWnd, message, wParam, lParam);
    }
    break;
case WM_PAINT:
    hdc = BeginPaint(hWnd, &ps);
for(y=0,it=v.begin();it!=v.end();it+=size.cy)

```

```

        if (iHscrollPos < it->length())
            TabbedTextOutA(hdc,0,y,it->data()+iHscrollPos,it->length()- iHscrollPos,0,NULL,0);
    EndPaint(hWnd, &ps);
    break;
    case WM_DESTROY: PostQuitMessage(0); break;
    default: return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}

```

Первое, что необходимо сделать для создания полос прокрутки — задать диапазон и установить позицию движка. Две функции `SetScrollRange()` и `SetScrollPos()` позволят решить эту задачу:

```

BOOL WINAPI SetScrollRange(HWND hWnd, int nBar, int nMinPos, int nMaxPos, BOOL bRedraw);
int WINAPI SetScrollPos(HWND hWnd, int nBar, int nPos, BOOL bRedraw);

```

- ❑ `hWnd` — дескриптор окна;
- ❑ `nBar` — идентификатор полосы прокрутки, здесь может принимать значение `SB_HORZ` или `SB_VERT`;
- ❑ `nMinPos`, `nMaxPos` — минимальное и максимальное значение позиции движка;
- ❑ `nPos` — текущая позиция движка;
- ❑ `bRedraw` — перерисовка полосы прокрутки, если значение поля `TRUE`, и нет перерисовки, если `FALSE`.

Удобнее всего установить параметры прокрутки в сообщении `WM_SIZE`, поскольку это сообщение обрабатывается при создании окна, и будет генерироваться системой при каждом изменении его размера.

Диапазон вертикальной прокрутки установим равным количеству прочитанных строк файла `n` за вычетом одной экранной страницы. Это желательно сделать, чтобы в конце текста не появлялась пустой страницы. Количество строк, помещающихся на одной странице, мы вычислим, поделив размер окна по вертикали на высоту строки `sy/size.cy`. Высоту окна получим здесь же, в сообщении `WM_SIZE`, извлекая старшее слово `lParam`, а геометрические параметры шрифта зададим при описании переменной: `static SIZE size = {8, 16}`;

Таким будет размер символа шрифта по умолчанию. Однако с такой арифметикой могут быть проблемы. Действительно, при создании окна переменная `n` равна 0, поскольку она объявлена как статическая и выражение: $k = n - sy/size.cy$;

будет отрицательным. Такой же результат будет и при открытии файла, который целиком помещается в одном окне. Поэтому мы в следующей строке скорректируем значение переменной `COUNT`, которая и будет определять диапазон вертикального скроллинга.

```

if (k > 0) COUNT = k; else COUNT = iVscrollPos = 0;

```

Заодно присваиваем нулевое значение переменной `iVscrollPos`, являющейся индексом текущей позиции вертикального скроллинга.

Теперь установим диапазон и позицию скроллинга:

```

SetScrollRange(hWnd, SB_VERT, 0, COUNT, FALSE);
SetScrollPos (hWnd, SB_VERT, iVscrollPos, TRUE);

```

ПРИМЕЧАНИЕ

Если минимальное и максимальное значения позиции движка совпадают, то полоса прокрутки в окне не выводится.

Параметры горизонтального скроллинга устанавливаются аналогично.

Очевидно, что при открытии приложения, когда количество строк `n` и максимальная длина строки `length` равны нулю, полос прокрутки не будет, они должны появиться лишь при открытии файла достаточно большого размера.

Немного модифицируем обработчик сообщения WM_CREATE, добавив в поле фильтра еще один шаблон: `file.lpstrFilter = _T("Text\0 *.txt\0Все файлы\0 *.*");`

Для корректной обработки команды **New** добавим две строки текста:

```
n = length = 0;
```

```
SendMessage(hWnd, WM_SIZE, 0, sy << 16 | sx);
```

Здесь мы обнулили переменные, описывающие количество строк файла и максимальную длину строки, после чего сгенерируем сообщение WM_SIZE при помощи функции `SendMessage()`, что вызовет переопределение параметров скроллинга:

```
LRESULT WINAPI SendMessageW(HWND hWnd, UINT Msg, WPARAM wParam, LPARAM  
lParam);
```

ПРИМЕЧАНИЕ

Этот прием генерации сообщения мы используем для экономии кода. Можно конечно оформить этот код отдельной функцией, но хотелось показать и эту возможность.

Функция `SendMessage()` обеспечивает передачу сообщения окну с дескриптором `hWnd`, дополнительная информация передается через `wParam` и `lParam`. Причем функция синхронная и возвращает управление только после обработки сообщения. Возвращаемое значение зависит от сообщения.

Поскольку в сообщении WM_SIZE мы получаем размеры окна в переменной `lParam`, то здесь мы должны самостоятельно сформировать его значение. В старшее слово `lParam` нужно поместить высоту окна — это мы сделаем операцией логического сдвига `sy<<16`, а ширину окна в младшее слово добавим операцией логического сложения. `wParam` здесь не используется, поэтому запишем 0.

При обработке команды **Open** вычислим максимальное количество символов в строке, эта информация нам понадобится для определения горизонтального скроллинга. Добавим в цикл чтения файла строку: `if (length < st.length()) length = st.length();`

По завершении чтения файла в переменной `length` будет храниться размер строки максимальной длины. Количество строк файла получим как размер контейнера: `n = v.size()`.

Перед тем как перерисовать окно, сгенерируем сообщение WM_SIZE для переустановки параметров скроллинга.

Теперь мы должны организовать логику работы с полосами прокрутки.

Любые действия с полосами прокрутки вызывают генерацию сообщения: WM_VSCROLL для вертикального и WM_HSCROLL для горизонтального скроллинга. В младшем слове `wParam` передается идентификатор команды. Мы будем обрабатывать лишь следующий набор команд:

- ☐ SB_LINEUP — щелчок мыши на стрелке вверх (влево);
- ☐ SB_LINEDOWN — щелчок мыши на стрелке вниз (вправо);
- ☐ SB_PAGEUP — щелчок мыши внутри полосы выше (левее) движка;
- ☐ SB_PAGEDOWN — щелчок мыши внутри полосы ниже (правее) движка;
- ☐ SB_THUMBPOSITION — отпускание движка после его буксировки.

ПРИМЕЧАНИЕ

При рассмотрении горизонтального скроллинга логичнее было бы использовать символические константы `SB_LINELEFT`, `SB_LINERIGHT`, `SB_PAGELEFT`, `SB_PAGERIGHT`; они эквивалентны приведенным выше и представляют собой целые числа: 0, 1, 2, 3.

Для отслеживания позиции движков полос прокрутки используем две статические переменные целого типа `iVscrollPos` — для вертикального скроллинга и `iHscrollPos` — для горизонтального. По командам `SB_LINEUP` и `SB_LINEDOWN` уменьшим или увеличим значение переменной на 1. По командам `SB_PAGEUP` и `SB_PAGEDOWN`, означающим перелистывание на страницу для горизонтального скроллинга, уменьшим или увеличим переменную `iVscrollPos` на количество помещающихся в окне строк, которое мы вычислим, поделив вертикальный размер окна на высоту строки: `sy/size.cy`. Для горизонтального скроллинга будем смещать переменную `iHscrollPos` на размер табуляции, равный 8 символам.

Последней обработаем команду SB_THUMBPOSITION, которая будет сгенерирована после отпущения движка по завершении его буксировки. Мы можем получить позицию движка из старшего слова wParam: `iVscrollPos = HIWORD(wParam)`.

Но если этим ограничиться, то будет возможно либо отрицательное значение, либо значение, превышающее максимальный размер скроллинга. Во избежание подобной ситуации воспользуемся следующей конструкцией: `iVscrollPos = max(0, min(iVscrollPos, COUNT))`;

`max()` и `min()` — макросы, определенные в файле включений `windows.h` для вычисления максимума и минимума двух арифметических выражений.

Теперь мы можем быть уверены, что переменная `iVscrollPos` не выйдет за пределы отрезка `[0; COUNT]`.

Затем проверяем, изменилась ли позиция движка? `if (iVscrollPos != GetScrollPos(hWnd, SB_VERT))`

Сравним значение переменной `iVscrollPos` с возвращаемым значением функции `GetScrollPos()` и, если значения не совпадают, перерисуем полосу скроллинга с новым положением движка:

```
SetScrollPos(hWnd, SB_VERT, iVscrollPos, TRUE);
```

После чего перерисуем содержимое окна, вызывая функцию `InvalidateRect()`.

Для горизонтального скроллинга выполним аналогичные действия.

Нам осталось рассмотреть организацию вывода текста в сообщении `WM_PAINT`. Выводом будут управлять две переменные `iVscrollPos` и `iHscrollPos`, определяющие позиции скроллинга. Будем начинать вывод текста со строки с индексом `iVscrollPos`, а горизонтальное смещение определим индексом `iHscrollPos`.

В цикле `for(...)` блок начальной инициализации будет выглядеть так: `y = 0, it = v.begin() + iVscrollPos`;

Переменная `y`, как и ранее, используется для вычисления `y`-координаты выводимой строки текста, а итератор `it`, которым мы выбираем строки текста из контейнера, сместим на `iVscrollPos` позиций. Для контейнера `vector` эта операция допустима и итератор сейчас показывает на строку текста с индексом `iVscrollPos`.

Рассмотрим, как записано условие: `it < v.end() && y < sy`

Цикл завершится, если будет достигнут конец контейнера или `y`-координата строки выйдет за пределы окна. Приращение координат:

```
++it, y += size.cy
```

Итератор `it` увеличивается на 1, а `y`-координата на высоту строки. Осталось вывести строку текста с учетом обработки табуляций:

```
TabbedTextOutA(hdc, 0, y, it->data() + iHscrollPos, it->length() - iHscrollPos, 0, NULL, 0);
```

Выводим строки текста со смещением `iHscrollPos`, размер строки уменьшается на эту же величину: `it->length() - iHscrollPos`.

ПРИМЕЧАНИЕ

Здесь мы поступаем не совсем корректно, поскольку выводим строки до конца, не заботясь о том, что часть строки может выйти за пределы окна. Это несколько ухудшает эффективность программы, но упрощает ее логику.

Панель инструментов

Чтобы получить максимально полное представление о возможностях Windows, можно создать для нашей задачи инструментальную панель, которая позволит выполнять команды меню нажатием на соответствующую кнопку. Для трех пунктов меню **New**, **Open** и **Save** создадим панель инструментов со стандартными для этих команд кнопками.

Традиционный путь создания панели инструментов заключается в вызове функции `CreateToolBarEx()`, прототип которой помещен в файл включений `commctrl.h`.

```
HWND WINAPI CreateToolBarEx(HWND hWnd, DWORD ws, UINT wID, int nBitmaps, HINSTANCE hBmInst, UINT wBMID, LPCTBBUTTON lpButtons, int iNumButtons, int dxButton, int
```

dyButton, int dxBitmap, int dyBitmap,UINT uStructSize); 13 параметров этой функции имеют следующее значение:

- HWND — описатель родительского окна;
- WS — флаги стиля окна;
- wID — идентификатор дочернего окна панели инструментов;
- nBitmaps — число изображений в битовом образе;
- hBMPInst — дескриптор ресурса битового образа для загрузки;
- wBMID — идентификатор ресурса битового образа;
- lpButtons — указатель на массив структур TBUTTON, которые содержат информацию о кнопках;
- iNumButtons — число кнопок инструментальной панели;
- dxButton — ширина кнопок в пикселах;
- dyButton — высота кнопок в пикселах;
- dxBitmap — ширина, в пикселах, кнопки битового образа;
- dyBitmap — высота, в пикселах, кнопки битового образа;
- uStructSize — размер структуры TBUTTON.

Информация о кнопках размещается в массиве структур TBUTTON.

struct TBUTTON

```
{
int iBitmap;           // индекс изображения кнопки
int idCommand;         // идентификатор команды, соответствующей кнопке
BYTE fsState;          // определяют начальное состояние и
BYTE fsStyle;          // стиль кнопки
BYTE bReserved[2];     // резерв
DWORD dwData;          // определенное приложением значение
int iString;           // индекс текстовой метки кнопки
};
```

Для описания включаемых в приложение кнопок в Windows имеется два набора готовых для использования битовых образов. Первый набор содержит изображения 15 кнопок, соответствующих командам меню **File** и **Edit**, откуда мы и позаимствуем необходимые изображения. Идентификаторы кнопок вполне отражают их назначение:

STD_CUT, STD_COPY, STD_PASTE, STD_UNDO, STD_REDO, STD_DELETE, STD_FILENEW,
STD_FILEOPEN, STD_FILESAVE, TD_PRINTPRE, STD_PROPERTIES,
STD_HELP, STD_FIND, STD_REPLACE, STD_PRINT.

Внесем в программу (листинг 2.2) необходимые изменения.

1. Добавим файл включения директивой:

```
#include <commctrl.h>
```

2. На глобальном уровне опишем 3 кнопки:

```
TBUTTON tbb[] ={
{STD_FILENEW, ID_FILE_NEW, TBSTATE_ENABLED, TBSTYLE_BUTTON,0,0,0,0},
{STD_FILEOPEN, ID_FILE_OPEN,TBSTATE_ENABLED, TBSTYLE_BUTTON,0,0,0,0},
{STD_FILESAVE, ID_FILE_SAVE,TBSTATE_ENABLED, TBSTYLE_BUTTON,0,0,0,0} };
```

Строка описания: имя кнопки, ее идентификатор, константа TBSTATE_ENABLED, которая означает, что кнопка активна; стиль кнопки стандартный — TBSTYLE_BUTTON, остальные 4 параметра здесь не используются, и мы заполним их нулями.

Поскольку мы выбрали для идентификаторов кнопок те же значения, что и для пунктов меню, нажатие на кнопку панели инструментов генерирует те же сообщения, что и команды меню — WM_COMMAND со значением идентификатора соответствующего пункта меню в младшем слове wParam. Таким образом, нет необходимости писать код для обработки сообщения о нажатии на кнопку панели инструментов, так как эти сообщения будут обработаны тем же кодом, что и команды меню.

3. Опишем дескриптор панели инструментов: static HWND hwndToolBar;

4. Теперь вызовем функцию `CreateToolBarEx()` и передадим ей необходимые параметры. Это можно сделать в сообщении `WM_CREATE`.

`hwndToolBar = CreateToolBarEx(hWnd, WS_CHILD | WS_VISIBLE | CCS_TOP, 1, 0, HINST_COMMCTRL, IDB_STD_SMALL_COLOR, tbb, 3, 0, 0, 0, 0, sizeof(TBBUTTON));` Стиль окна определяется логической суммой трех констант: `WS_CHILD` — окно является дочерним; `WS_VISIBLE` — окно отображается автоматически (если не включить этот стиль, то необходимо использовать функцию `ShowWindow()`); `CCS_TOP` — окно размещается сверху под строкой меню, причем размер по вертикали устанавливается исходя из размеров кнопок, а горизонтальный размер устанавливается по ширине родительского окна.

Следующий параметр — идентификатор окна панели инструментов, присваиваем ему номер 1. Далее 0, поскольку для стандартного ресурса `HINST_COMMCTRL` этот параметр не используется. Идентификатор ресурса выберем `IDB_STD_SMALL_COLOR` для "маленьких" (16 16) кнопок. Далее описывается `tbb` — указатель на массив структур `TBBUTTON` и количество элементов — 3. Следующие 4 поля для стандартных элементов управления не используются и заполняются нулевым значением, а последним параметром стоит размер структуры `sizeof(TBBUTTON)`.

ПРИМЕЧАНИЕ

Следует иметь в виду, что для хранения изображения всех элементов панели инструментов используется один битовый образ, а извлечение необходимого образа происходит по индексу.

5. Необходимо предусмотреть изменение размеров панели инструментов при изменении размеров окна, иначе панель инструментов будет выглядеть не очень привлекательно. Лучше всего сделать это при обработке сообщения `WM_SIZE` посылкой сообщения `TB_AUTOSIZE`:

`SendMessage(hwndToolBar, TB_AUTOSIZE, 0, 0);`

что приведет к автоматической корректировке размера панели инструментов, и она гарантированно будет занимать всю верхнюю строку.

6. Теперь осталось решить последнюю проблему: дело в том, что панель инструментов, являясь дочерним окном, лежит внутри клиентской области окна и при выводе текста перекрывает его верхнюю часть. Можно решить эту проблему, сместив начальную точку вывода текста. При обработке сообщения `WM_SIZE` вычислим высоту панели инструментов при помощи функции `GetWindowRect()`, которая возвращает координаты окна относительно рабочего стола:

`GetWindowRect(hwndToolBar, &rt); size_Toolbar = rt.bottom - rt.top;`

переменные `rt` и `size_Toolbar` предварительно определим в оконной функции `static int size_Toolbar;`

`RECT rt;`

В сообщении `WM_PAINT` при начальной инициализации цикла переменной `y` зададим значение: `for(y = size_Toolbar,...),`

при этом первая строка текста будет смещена ниже панели инструментов. Однако в этом случае необходимо также скорректировать и размах скроллинга по вертикали, поскольку он зависит от размеров окна. В сообщении `WM_SIZE` отредактируем строку: `k = n - (sy - size_Toolbar)/size.cy;`

7. Мы все сделали правильно, но при компиляции будет выдано сообщение об ошибке: `error LNK2001: unresolved external symbol __imp__CreateToolBarEx@52`

Дело в том, что библиотека, в которой находится тело функции `CreateToolBarEx()`, по умолчанию не подключается к проекту. Необходимо сделать это "вручную". Откроем окно свойств проекта в диалоговом окне **Solution Explorer**. На подложке **Linker | Input** в окне редактирования **Additional Dependencies** добавляем к списку библиотечных файлов `comctl32.lib`.

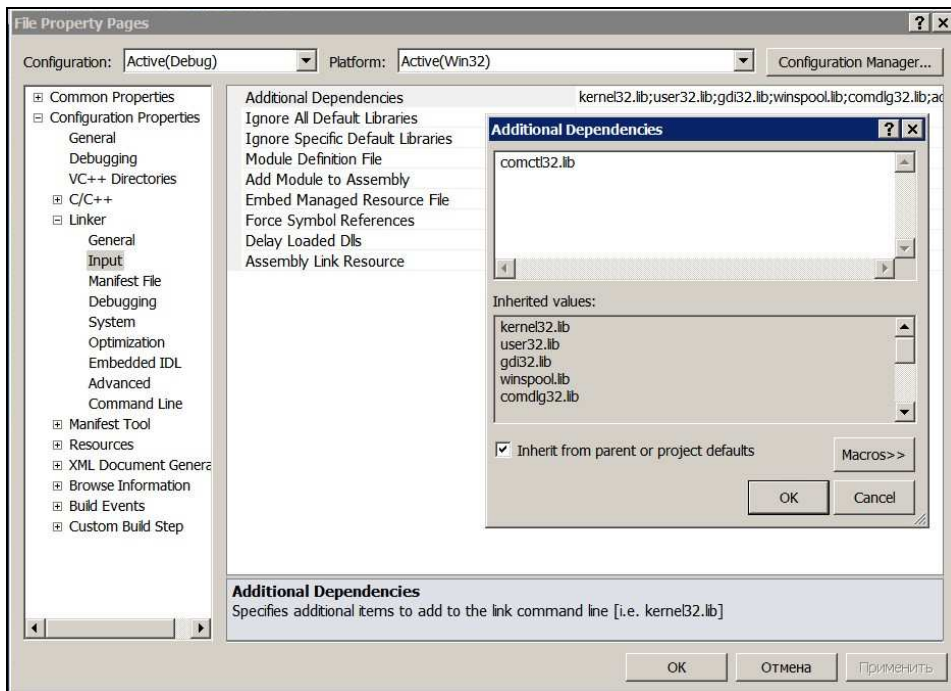


Рис. 2.3. Диалоговое окно настройки параметров проекта

ПРИМЕЧАНИЕ

Имя файла `comctl32.lib` выглядит не очень красиво, но это наследие операционной системы MS-DOS, где имя файла не превышало 8 символов.

Теперь компиляция должна пройти успешно. На рис. 2.4 показан пример работы созданного приложения.

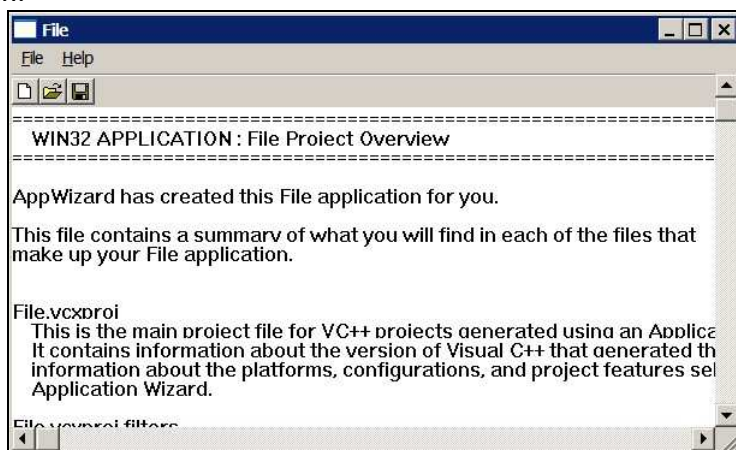


Рис. 2.4. Работа приложения с панелью инструментов

Выбор шрифтов

Можно и далее наращивать функциональные возможности программы просмотра файлов. Следующее расширение позволит программе произвольно выбирать шрифт для вывода текста. Воспользуемся имеющимся в системе стандартным диалогом выбора шрифта.

Для обращения к диалоговому окну выбора шрифта необходимо описать две переменные типа `CHOOSEFONT` и `LOGFONT`, где первая обеспечивает взаимодействие с функцией диалога, а вторая требуется для хранения информации о выбранном шрифте. Обе структуры имеют большое количество полей и определены в файлах включений `commdlg.h` и `wingdi.h`.

Финальный текст оконной функции приведен в листинге 2.3, здесь же обсудим лишь новые элементы. Начнем с описания переменных:

```
static LOGFONT lf;
static CHOOSEFONT cf;
static HFONT hfont;
```

Нам потребуется определить лишь 4 поля структуры `CHOOSEFONT`. Сделаем это в сообщении `WM_CREATE`:


```
cf.lStructSize = sizeof(CHOOSEFONT);  
cf.Flags = CF_EFFECTS | CF_INITTODLOGFONTSTRUCT | CF_SCREENFONTS;  
cf.hwndOwner = hWnd; cf.lfLogFont = &lf;
```

Первое поле содержит размер структуры CHOOSEFONT. Следующие поля определяют флаги и дескриптор родительского окна, а последнее поле хранит указатель на переменную типа LOGFONT.

Flags задает режим работы диалогового окна:

- ❑ CF_EFFECTS — позволяет пользователю определять зачеркнутый, подчеркнутый шрифт, а также цвет текста;
- ❑ CF_INITTODLOGFONTSTRUCT — используется структура LOGFONT;
- ❑ CF_SCREENFONTS — выводится список только экранных шрифтов, поддерживаемых системой.

Теперь можно строить обращение к функции диалога ChooseFont():

```
BOOL APIENTRY ChooseFont(LPCHOOSEFONT);
```

Функция принимает указатель на переменную типа CHOOSEFONT и возвращает TRUE при успешной работе диалога.

Добавим в выпадающее меню **Оформление** пункт **Шрифт** с идентификатором ID_FONT и построим его обработчик.

ПРИМЕЧАНИЕ

Перед открытием окна **Menu** нужно установить в поле свойств **Language** — Русский (Россия).

После выбора шрифта мы, чтобы "не замусорить" память, уничтожим предыдущий шрифт, если он существовал, при помощи конструкции if (hfont) ...

Затем создадим шрифт функцией CreateFontIndirect().

В качестве параметра функция принимает указатель на структуру LOGFONT и заполняет ее поля в ходе диалога. Теперь необходимо определить метрику текста: нам нужна высота шрифта и средняя ширина символа. Определить эти параметры можно обращением к функции GetTextMetrics(), которую мы уже рассматривали в *главе 1*. Однако предварительно необходимо получить контекст устройства hdc и выбрать созданный шрифт текущим. Среднюю ширину символа мы получим из поля tmAveCharWidth структуры TEXTMETRIC, которую также нужно предварительно описать:

```
TEXTMETRIC tm;
```

Высоту строки дает выражение: `tm.tmHeight + tm.tmExternalLeading`.

Осталось переопределить параметры скроллинга, послав сообщение WM_SIZE:

```
SendMessage(hWnd, WM_SIZE, 0, sy << 16 | sx);
```

и перерисовать окно вызовом функции InvalidateRect().

В сообщении WM_PAINT необходимо добавить код для переключения на выбранный шрифт и установить его цвет.

И последний штрих. Практически все современные манипуляторы типа "мышь" имеют среднее колесико, которое обычно используется для прокрутки текста. Было бы уместно и нам воспользоваться этой возможностью, тем более, что это довольно просто.

Прокрутка колесика мыши порождает сообщение WM_MOUSEWHEEL, а в старшем слове wParam возвращается количество шагов колесика, умноженное на константу WHEEL_DELTA (значение константы 120). Нам достаточно поделить это значение на данную константу, и мы получим количество шагов. Позаботимся о том, чтобы не выйти за границы скроллинга, установим новую позицию движка и перерисуем окно.

```
case WM_MOUSEWHEEL:
```

```
    iVscrollPos -= (short)HIWORD(wParam)/WHEEL_DELTA;    iVscrollPos = max(0,  
min(iVscrollPos, COUNT));
```

```
    SetScrollPos(hWnd, SB_VERT, iVscrollPos, TRUE);
```

```
InvalidateRect(hWnd, NULL, TRUE);    break;
```


ПРИМЕЧАНИЕ

Принципиально важно задать явное преобразование типа (short)HIWORD(wParam), иначе отрицательное значение в старшем слове wParam будет интерпретироваться как очень большое положительное число.

Листинг 2.3. Окончательный вариант оконной функции программы просмотра файлов

```
#include <commdlg.h>
#include <fstream>
#include <vector>
#include <string>
#include <commctrl.h>
TBBUTTON tbb[] =
{
    {STD_FILENEW, ID_FILE_NEW, TBSTATE_ENABLED, TBSTYLE_BUTTON, 0, 0, 0, 0},
    {STD_FILEOPEN, ID_FILE_OPEN, TBSTATE_ENABLED, TBSTYLE_BUTTON, 0, 0, 0, 0},
    {STD_FILESAVE, ID_FILE_SAVE, TBSTATE_ENABLED, TBSTYLE_BUTTON, 0, 0, 0, 0} };
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    PAINTSTRUCT ps;
    HDC hdc;
    static TCHAR name[256] = _T("");;
    static OPENFILENAME file;
    std::ifstream in;
    std::ofstream out;
    static std::vector<std::string> v;
    std::vector<std::string>::iterator it;
    std::string st;
    int y, k;
    static int n, length, sx, sy, cx, iVscrollPos, iHscrollPos, COUNT, MAX_WIDTH;
    static SIZE size = {8, 16 }; static HWND hWndToolBar;
    static int size_Toolbar; RECT rt;
    static LOGFONT lf; static CHOOSEFONT cf;
    static HFONT hfont; TEXTMETRIC tm;
    switch (message)
    {
        case WM_CREATE:
            file.lStructSize = sizeof(OPENFILENAME);
            file.hInstance = hInst;
            file.lpstrFilter = _T("Text .txt\0 *.txt\0Все файлы\0 *.*");
            file.lpstrFile = name;
            file.nMaxFile = 256;
            file.lpstrInitialDir = _T(".\\");
            file.lpstrDefExt = _T("txt");
            hWndToolBar = CreateToolBarEx(hWnd, WS_CHILD | WS_VISIBLE | CCS_TOP, 1, 0,
HINST_COMMCTRL, IDB_STD_SMALL_COLOR, tbb, 3, 0, 0, 0, 0, sizeof(TBBUTTON));
            cf.lStructSize = sizeof(CHOOSEFONT);
            cf.Flags = CF_EFFECTS | CF_INITTOLOGFONTSTRUCT | CF_SCREENFONTS;
            cf.hwndOwner = hWnd;
            cf.lpLogFont = &lf; break;
        case WM_SIZE:
            sx = LOWORD(lParam);
            sy = HIWORD(lParam);
```

```

k = n - (sy - size_Toolbar)/size.cy;
if (k > 0) COUNT = k; else COUNT = iVscrollPos = 0;
SetScrollRange(hWnd, SB_VERT, 0, COUNT, FALSE);
SetScrollPos(hWnd, SB_VERT, iVscrollPos, TRUE);
k = length - sx/size.cx;
if (k > 0) MAX_WIDTH = k; else MAX_WIDTH = iHscrollPos = 0;
SetScrollRange(hWnd, SB_HORZ, 0, MAX_WIDTH, FALSE);
SetScrollPos(hWnd, SB_HORZ, iHscrollPos, TRUE);
SendMessage(hWndToolBar, TB_AUTOSIZE, 0, 0);
GetWindowRect(hWndToolBar, &rt);
size_Toolbar = rt.bottom - rt.top;
break;
    case WM_MOUSEWHEEL:
        iVscrollPos -= (short)HIWORD(wParam)/WHEEL_DELTA;
        iVscrollPos = max(0, min(iVscrollPos, COUNT));
        SetScrollPos(hWnd, SB_VERT, iVscrollPos, TRUE);
InvalidateRect(hWnd, NULL, TRUE);
break;
case WM_VSCROLL :
    switch(LOWORD(wParam))
    {
    case SB_LINEUP : iVscrollPos--; break;
    case SB_LINEDOWN : iVscrollPos++; break;
    case SB_PAGEUP : iVscrollPos -= sy/size.cy; break;
    case SB_PAGEDOWN : iVscrollPos += sy/size.cy; break;
    case SB_THUMBPOSITION :
        iVscrollPos = HIWORD(wParam); break;
    }
    iVscrollPos = max(0, min(iVscrollPos, COUNT));
    if (iVscrollPos != GetScrollPos(hWnd, SB_VERT))
    {
        SetScrollPos(hWnd, SB_VERT, iVscrollPos, TRUE);
        InvalidateRect(hWnd, NULL, TRUE);
    }
    break;
case WM_HSCROLL :
    switch(LOWORD(wParam))
    {
    case SB_LINEUP : iHscrollPos--; break;
    case SB_LINEDOWN : iHscrollPos++; break;
    case SB_PAGEUP : iHscrollPos -= 8; break;
    case SB_PAGEDOWN : iHscrollPos += 8; break;
    case SB_THUMBPOSITION : iHscrollPos = HIWORD(wParam); break;
    }
    iHscrollPos = max(0, min(iHscrollPos, MAX_WIDTH));
    if (iHscrollPos != GetScrollPos(hWnd, SB_HORZ))
    {
        SetScrollPos(hWnd, SB_HORZ, iHscrollPos, TRUE);
        InvalidateRect(hWnd, NULL, TRUE);
    }
    break;

```

```

case WM_COMMAND:
    switch (LOWORD(wParam))
    {
        case ID_FILE_NEW :
            if (!v.empty()) std::vector<std::string>().swap(v);
            n = length = 0;
            SendMessage(hWnd, WM_SIZE, 0, sy << 16 | sx);
            InvalidateRect(hWnd, NULL, TRUE); break;
        case ID_FILE_OPEN :
            file.lpszTitle = _T("Открыть файл для чтения");
            file.Flags = OFN_HIDEREADONLY;
            if (!GetOpenFileName(&file)) return 1;
            in.open(name);
            while (getline(in, st))
            {
                if (length < st.length()) length = st.length();    v.push_back(st);
            }
            in.close();    n = v.size();
            SendMessage(hWnd, WM_SIZE, 0, sy << 16 | sx);
            InvalidateRect(hWnd, NULL, 1);
            break;
        case ID_FILE_SAVE :
            file.lpszTitle = _T("Открыть файл для записи");
            file.Flags = OFN_NOTESTFILECREATE;
            if (!GetSaveFileName(&file)) return 1;    out.open(name);
            for (it = v.begin(); it != v.end(); ++it) out << *it << '\n';    out.close();    break;
        case ID_FONT :
            if(ChooseFont(&cf))
            {
                if (hfont) DeleteObject(hfont);
                hfont = CreateFontIndirect(&lf);
                hdc = GetDC(hWnd);
                SelectObject(hdc, hfont);
                GetTextMetrics(hdc, &tm);
                size.cx = tm.tmAveCharWidth;
                size.cy = tm.tmHeight + tm.tmExternalLeading;
                ReleaseDC(hWnd, hdc);
                SendMessage(hWnd, WM_SIZE, 0, sy << 16 | sx);
                InvalidateRect(hWnd, NULL, TRUE);
            }
            break;
        case IDM_EXIT: DestroyWindow(hWnd); break;
        default: return DefWindowProc(hWnd, message, wParam, lParam);
    } break; case WM_PAINT:
        hdc = BeginPaint(hWnd, &ps);
        if (hfont)
        { SelectObject(hdc, hfont);
          SetTextColor(hdc, cf.rgbColors);
        }
        for (y = size_Toolbar, it = v.begin() + iVscrollPos;    it != v.end() && y < sy; ++it, y += size.cy)
            if (iHscrollPos < it->length())

```

```

        TabbedTextOutA(hdc, 0, y, it->data()+iHscrollPos,      it->length()-iHscrollPos, 0, NULL, 0);
EndPaint(hWnd, &ps); break; case WM_DESTROY:
    if (hfont) DeleteObject(hfont); PostQuitMessage(0); break;
    default: return DefWindowProc(hWnd, message, wParam, lParam);
}
return 0;
}

```

Пожалуй, это все, что можно было сделать на данном этапе (рис. 2.5).



Рис. 2.5. Окно финальной версии программы-просмотрщика текстовых файлов

Однако мы должны отдавать себе отчет, что не все проблемы нашли удовлетворительное решение:

1. Так, максимальная длина строки, необходимая для организации горизонтального скроллинга, определена не совсем корректно при наличии в тексте символов табуляции.
2. Нам пришлось выводить текст до конца строки, даже если он выходит за границу окна, поскольку решение этой проблемы доступными на данном уровне средствами требует слишком много усилий.
3. Горизонтальный скроллинг будет работать удовлетворительно для моноширинных шрифтов, но для TrueType, шрифтов, когда ширина символов различна, строки текста будут "плавать" при скроллинге, и мы можем потерять "хвосты" длинных строк.

Если первые две проблемы можно как-то решить, написав дополнительный код, то в последнем случае у нас пока нет подходящего механизма.

Наиболее просто эти проблемы разрешаются при использовании виртуальных окон, о чем мы будем говорить в *главе 4*.

Чтение и запись файлов в библиотеке Win32 API

В Win32 API имеется свой набор функций, обеспечивающих работу с файлами. Мы не будем долго задерживаться на этом вопросе, поскольку нас пока вполне устраивает библиотека потоковых классов, обеспечивающая достаточно хороший сервис. Однако для полноты картины посмотрим, как можно создать программу чтения и записи текстовых файлов, используя имеющийся набор API-функций (листинг 2.4).

Нам понадобятся 3 функции:

1. Для открытия или создания файла используется функция:

```

HANDLE WINAPI CreateFileW(
    LPCWSTR lpFileName,          //имя файла
    DWORD dwDesiredAccess,       //способ доступа
    DWORD dwShareMode,           //совместный доступ
    LPSECURITY_ATTRIBUTES lpSecurityAttributes, //атрибуты доступа
    DWORD dwCreationDisposition, //проверка существования
    DWORD dwFlagsAndAttributes,  //атрибуты файла

```

HANDLE hTemplateFile); //дескриптор временного файла 2. Читать из файла
будем функцией:

```
BOOL WINAPI ReadFile(  
HANDLE hFile,                //дескриптор файла  
LPVOID lpBuffer,             //буфер в памяти  
DWORD nNumberOfBytesToRead,  //максимальное число байтов  
LPDWORD lpNumberOfBytesRead, //указатель переменной, возвращающей  
                             //количество фактически прочитанных байтов  
LPOVERLAPPED lpOverlapped); //указатель на структуру OVERLAPPED
```

3. Пишем в файл функцией:

```
BOOL WINAPI WriteFile(  
HANDLE hFile,                //дескриптор файла  
LPCVOID lpBuffer,            //буфер в памяти  
DWORD nNumberOfBytesToWrite, //число записываемых байтов  
LPDWORD lpNumberOfBytesWritten, //указатель переменной,  
                                //возвращающей количество фактически  
                                записанных байтов  
LPOVERLAPPED lpOverlapped); //указатель на структуру OVERLAPPED
```

Более подробную информацию об этих функциях можно почерпнуть в *MSDN* (Microsoft Developer Network), а функцию `CreateFile()` мы рассмотрим подробнее в *главе 6*.

Листинг 2.4. Организация чтения/записи файла в библиотеке API-функций

```
#include <commdlg.h> const DWORD  
MaxLength = 0x7fff;  
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM  
lParam) {  
    PAINTSTRUCT ps; HDC hdc; static TCHAR  
name[256] = _T("");; static OPENFILENAME file; DWORD  
result; static HANDLE hFile; static char text[MaxLength];  
static int sx, sy; static DWORD nCharRead; RECT rt; switch  
(message)  
{  
    case WM_CREATE:  
        file.lStructSize = sizeof(OPENFILENAME); file.hInstance = hInst;  
file.lpstrFilter = _T("Text\0*.txt\0Все файлы\0*."); file.lpstrFile = name;  
file.nMaxFile = 256; file.lpstrInitialDir = _T(".\\"); file.lpstrDefExt = _T("txt");  
break; case WM_SIZE:  
    sx = LOWORD(lParam); sy = HIWORD(lParam);  
    break; case WM_COMMAND:  
        switch (LOWORD(wParam))  
        {  
            case ID_FILE_NEW :    nCharRead = 0;  
InvalidRect(hWnd, NULL, TRUE);  
break; case ID_FILE_OPEN :  
            file.lpstrTitle = _T("Открыть файл для чтения"); file.Flags =  
OFN_HIDEREADONLY; if (!GetOpenFileName(&file)) return 1; hFile =  
CreateFile(name, GENERIC_READ, 0, NULL, OPEN_EXISTING,  
FILE_ATTRIBUTE_NORMAL, NULL);  
ReadFile(hFile, text, MaxLength, &nCharRead, NULL); CloseHandle(hFile);  
if (nCharRead == MaxLength)  
            {  
                MessageBox(hWnd, _T("Слишком большой файл"),
```

```

        _T("Неудачное открытие файла"), MB_YESNO | MB_ICONHAND);
    return 0;
}
InvalidateRect(hWnd, NULL, TRUE);
break; case ID_FILE_SAVE :
    file.lpszTitle = _T("Открыть файл для записи"); file.Flags =
OFN_NOTESTFILECREATE; if (!GetSaveFileName(&file)) return 1; hFile =
CreateFile(name, GENERIC_WRITE, 0, NULL, CREATE_ALWAYS,
FILE_ATTRIBUTE_NORMAL, NULL);
    WriteFile(hFile, text, nCharRead, &result, NULL); CloseHandle(hFile);
break;
case IDM_EXIT: DestroyWindow(hWnd); break; default: return
DefWindowProc(hWnd, message, wParam, lParam);
}
break; case WM_PAINT:
    hdc = BeginPaint(hWnd, &ps);
    SetRect(&rt, 0, 0, sx, sy);
    DrawTextA(hdc, text, nCharRead, &rt, DT_LEFT);
EndPaint(hWnd, &ps); break;
case WM_DESTROY: PostQuitMessage(0); break; default: return
DefWindowProc(hWnd, message, wParam, lParam);
}
return 0;
}

```

Диалог открытия файла для чтения и записи позаимствуем у предыдущей задачи (см. листинг 2.1), отсюда же возьмем обработчики сообщений WM_CREATE и WM_SIZE. Открываем файл для чтения функцией CreateFile():

```
hFile = CreateFile(name, GENERIC_READ, 0, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
```

Первый параметр name — полное имя файла. GENERIC_READ означает, что файл открывается для чтения. Следующий параметр 0 — совместного доступа к файлу нет. NULL — атрибуты доступа наследуются от порождающего процесса. OPEN_EXISTING — проверка существования файла. FILE_ATTRIBUTE_NORMAL — файл с "нормальными" атрибутами (т. е. файл не объявлен, как скрытый, системный и т. д.) и последний параметр NULL — временный файл не создается.

Возвращаемое значение функции hFile — дескриптор файла. Здесь было бы уместно проверить, насколько удачно завершилась операция открытия файла. Если файл не смог открыться, функция возвращает 0.

```
Читаем файл одним блоком функцией ReadFile(): ReadFile(hFile, text, MaxLength,
&nCharRead, NULL);
```

Здесь hFile — дескриптор открытого файла. text — указатель символьного массива, куда считывается максимально MaxLength байтов. В переменной nCharRead мы получим фактическое число прочитанных байтов. Последний параметр NULL означает, что нет перекрытия читаемого блока данных.

Здесь мы для упрощения задачи ограничились максимальным размером файла в MaxLength = 0x7fff байтов. Вообще-то функция может читать блоки размером до 64 К. Смысл нашего ограничения будет понятен, когда мы рассмотрим элементы управления в *главе 3*.

Если же размер файла окажется больше, то будет прочитан блок в MaxLength байтов, а nCharRead вернет это же значение. В этом случае мы в диалоговом окне выводим сообщение **"Слишком большой файл"**.

При записи открываем файл также функцией CreateFile():

```
hFile = CreateFile(name, GENERIC_WRITE, 0, NULL, CREATE_ALWAYS,
```



```
FILE_ATTRIBUTE_NORMAL, NULL);
```

где вторым параметром стоит идентификатор `GENERIC_WRITE`, означающий, что файл открыт для записи, а параметр `CREATE_ALWAYS` означает, что создается новый файл, существующий файл будет уничтожен.

Записываем в файл также одним блоком функцией `WriteFile()`:

```
WriteFile(hFile, text, nCharRead, &result, NULL);
```

где все параметры имеют тот же смысл, что и для функции `ReadFile()`. Нам пришлось описать переменную `result`, в которую возвращается количество записанных байтов. Эту переменную можно использовать для контроля правильности записи. Если блок данных записан на диск, то ее значение совпадает с `nCharRead`.

Как при чтении, так и при записи, закрываем файл функцией `CloseHandle()`. Поскольку в этой задаче мы работаем с символьным массивом, а не C-строкой, то в меню **New** достаточно обнулить число прочитанных байтов и перерисовать окно: `nCharRead = 0;`

При выводе содержимого файла в сообщении `WM_PAINT` возникают определенные сложности. Дело в том, что мы прочитали файл как байтовый массив, имеющий ту же структуру, что и текстовый файл, в частности в конце строки присутствует пара символов — `'\r\n'`. Можно, конечно, "разобрать" файл на строки и выводить в окно построчно, но можно поступить проще, воспользовавшись функцией `DrawText()`, которая и предназначена для вывода текстового массива такой структуры. Предварительно только нужно создать прямоугольник размером с окно:

```
SetRect(&rt, 0, 0, sx, sy);
```

Текст выведем следующей конструкцией:

```
DrawTextA(hdc, text, nCharRead, &rt, DT_LEFT);
```

 где мы установили

выравнивание текста влево.

Мы не будем останавливаться на вопросах организации скроллинга. Предложим эту задачу для самостоятельной работы.

Вопросы.